LabGenius S Automated Material Handling Robot Software User Manual



Please read the entire software user manual before you start.

- Version: V1.0
- Models: S4-1.5-0.4-Z0.4, S4-1.5-0.6-Z0.6, S4-1.5-0.6-Z0.8

Bioyond Robotics (Shanghai) Co., Ltd

Contents

1. PREFACE 3

1.1. COPYRIGHT STATEMENT 3
1.2. SCOPE OF APPLICATION 3

2. SOFTWARE INTRODUCTION 4

3. QUICK START 5

3.1. SOFTWARE INSTALLATION 5
3.2. POWER ON 5
3.3. LOGIN 6
3.4. CONNECTION 6
3.5. LOGIN 7
3.6. EXECUTION PROGRAM 7

4. HOME PAGE 9

4.1. MAIN MENU 9
4.2. MAIN WINDOW 9
4.3. LOG BAR 9
4.4. STATUS BAR 10

5. JOG CONTROL PANEL 12

5.1. EMERGENCY STOP 12
5.2. CLEAR ERROR 13
5.3. MODE SELECTION 13
5.4. DRAGGO 14
5.5. COMMON TOOLS 14
5.6. AUTO/MANUAL CONTROL 15
5.7. JOG CONTROL 15

6. PROGRAM 17

6.1. PROGRAM LIBRARY 17
6.2. PROJECT MANAGEMENT 18
6.3. SUBROUTINE MANAGEMENT 20
6.4. CODING ASSISTANCE 22
6.5. SHORTCUT KEY 24
6.6. PROGRAM EXECUTION 24

7. MODULE 27

7.1. UPDATE FIRMWARE 27
7.2. CONFIGURE NETWORK 28

8. SETTING 29

8.1. USER 29
8.2. USER GROUP 30
8.3. COORDINATE AND LOAD 31
8.4. SYSTEM SETTING 40

8.5. BACKUP AND RESTORE 42

9. USER SETTING 44

APPENDIX A: COMMANDS AND DATA TYPES 45

A.1 GENERIC VARIABLE 45
A.2 MOVE COMMAND 46
A.3 LOGICAL COMMAND 49
A.4 SYSTEM COMMAND 50
A.5 IO COMMAND 54
A.6 OPERATION COMMAND 56
A.7 BIT OPERATION COMMAND 57
A.8 SYSTEM VARIABLE COMMAND 58
A.9 STACK COMMAND 59
A.10 GRIPPER CONFIGURATION COMMAND 60
A.11 THIRDPARTY GRIPPER COMMAND 61

1. Preface

Welcome to the Bioyond Robotics Automated Material Handling Robot.
Thank you for choosing our product. We appreciate your trust in Bioyond Robotics.
This manual provides comprehensive guidance on setting up and operating the LabGenius S4 robot, emphasizing topics most relevant to its everyday use in laboratory environments. To ensure safe and effective operation, please read this manual thoroughly before using the system.
Need assistance? Contact our support team for any questions or concerns. We're here to help!

1.1. Copyright Statement

Copyright Notice
© 2024 Bioyond Robotics. All rights reserved.
This document and its contents, including but not limited to text, images, and graphics, are protected by copyright and other intellectual property laws. No part of this document may be reproduced, distributed, or transmitted in any form or by any means, electronic or mechanical, without the prior written permission of Bioyond Robotics.

Disclaimer
The information contained in this manual is provided "as is" and without warranty of any kind, express or implied. Bioyond Robotics disclaims all warranties, including but not limited to implied warranties of merchantability and fitness for a particular purpose. Bioyond Robotics shall not be liable for any damages, including but not limited to direct, indirect, incidental, special, or consequential damages, arising out of or in connection with the use of this manual or the product described herein.

Product Information and Updates
The product specifications and software/hardware configurations may change without notice. For the most current information, please check the factual product information carefully before reading this maual.

Errors and Omissions
Bioyond Robotics strives to ensure the accuracy and completeness of this manual. However, there may be errors or omissions. If you encounter any discrepancies or have questions, please contact our support team.

Modifications
Bioyond Robotics reserves the right to modify the product and this manual at any time without notice.

1.2. Scope of Application

The information provided in this manual is specifically designed for the Bioyond Robotics automated material handling robot and its accompanying software components.

2. Software Introduction

Bioyond Works is the companion software designed for the LabGenius S automated material handling robot. It offers a comprehensive suite of tools, including manual robot control, program editing and execution, module management, and system configuration.
This manual provides detailed instructions on using Bioyond Works, covering:

- Software installation and setup.
- Device connection and communication.
- Editing and managing robot work programs.
- System management and customization.

Bioyond Works provides two operating modes: Offline Mode and Online Mode.

Offline Mode: Only limited functionalities are available in the Offline Mode:

- Edit offline projects
- View partial module information
- User management

⚠Warning: Users cannot perform device control, run programs, or configure the system in the Offline Mode.

Online Mode: Once the software successfully connects to the robot, it enters Online Mode enabling users to gain access to all software functionalities, including:

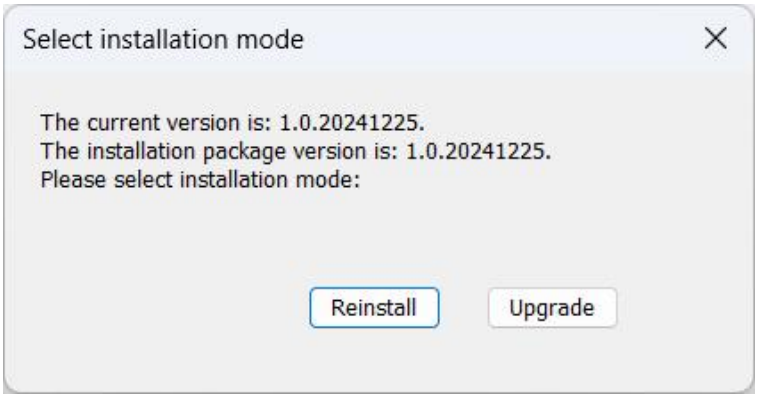
- Jog control of the robot
- Edit both online and offline projects
- Execute online projects
- View all module information
- User management
- Define the robot coordinate systems and loads
- Configure system parameters
- Backup and restore controller's data

3. Quick Start

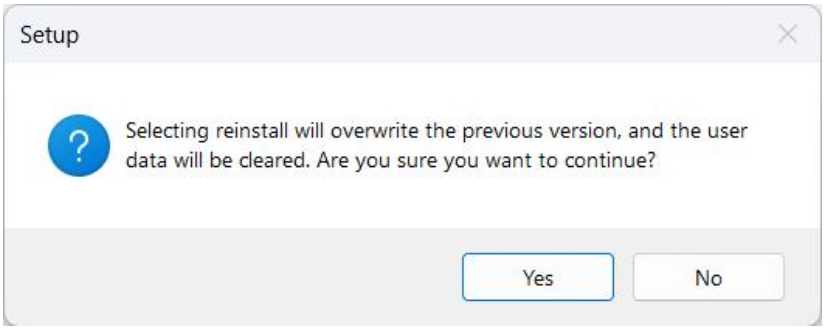
3.1. Software Installation

Bioyond Works requires Windows 10 or higher operating system. Bioyond Works is packaged as an installer. Follow these steps to use it:

- Run the BioyondWorks_Robot.exe file by double-clicking it.
- During the installation, choose one of the following modes:
  Reinstall: Recommanded for first-time installation or when installing on a new system.
  Upgrade: Use this option when updating an existing installation.



⚠ Warning: In the reinstall mode, the previous version will be overwritten and user data will be cleared. In the subsequent installation, run the "Upgrade" command.



- Follow the instructions in the installer to complete the installation. After the installation is finished, the application will automatically launch. If prompted by the system, grant the application security or firewall permissions to ensure it can start and communicate with the controller over the network.

3.2. Power On

Before starting the robot , you need to complete the following preparations:

- Remove the screws fixed during transportation and install the robot correctly.
- Check that the power supply is normal, and plug the AC power plug into the power socket.

- Side Cable Outlet Model: Turn on the main power switch on the side of the robotic arm's pillar to power on the robot.

  Bottom Cable Outlet Model: The robot powers on automatically when the external power supply is turned on, with no further action required.

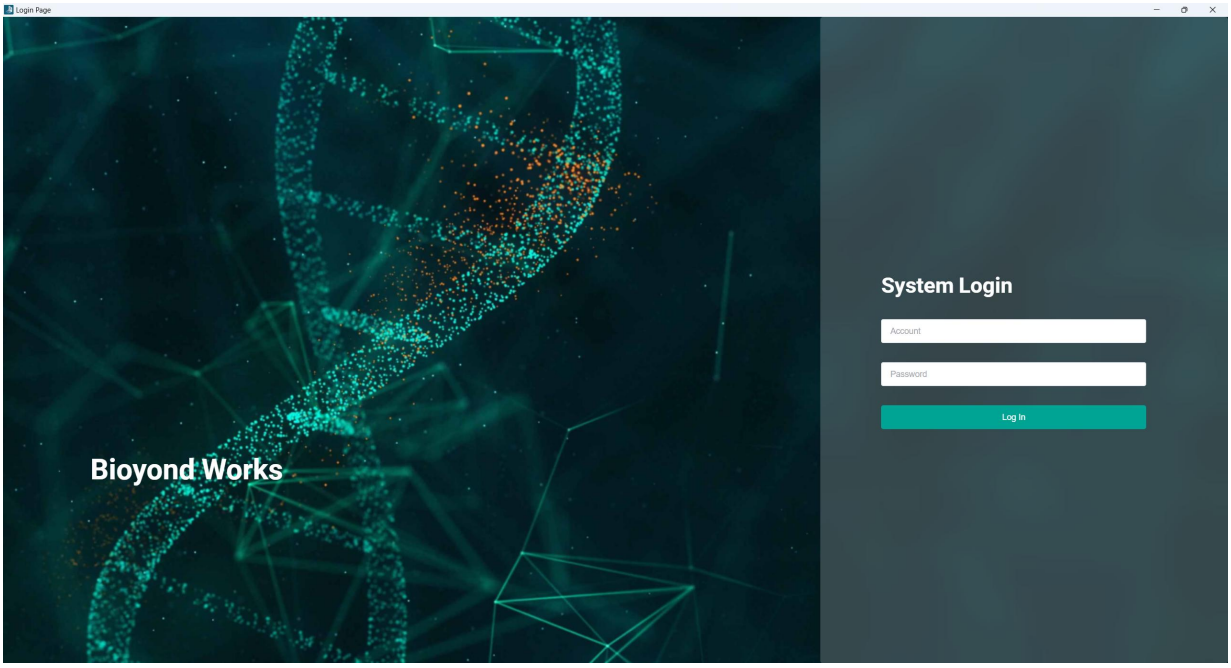
3.3. Login

After the PC connects to the workstation, double-click the Bioyond Works-S icon created on the desktop during installation to launch the software. Once launched, the user login page will be displayed.

The default login account is **admin** and password is **Admin@123** .

⚠Warning: You can change your password after logging in. Please manage your password properly to avoid losing it.

After entering your credentials, you will access the software homepage,where you can begin using the software.



3.4. Connection

Bioyond Works software supports connecting to the robot using either a wired (Ethernet) and wireless (Wi-Fi) connection. The specific operation steps are as follows:

Wired connection (Ethernet):

- Connect the computer with Bioyond Works installed to robot via Ethernet cable.
- Set the computer's IP address to 192.168.3.x(ensure it is not same as the controller's IP).
- Open the Bioyond Works, select “Module” in the menu bar, set the IP address to 192.168.3.200 in the network configuration page.
- Bioyond Works will automatically establish communication with robot .

Wireless connection (Wi-Fi):

- Open the computer's Wi-Fi settings and choose the robot's Wi-Fi.
- Open the Bioyond Works, select “Module” in the menu bar, set the IP address to 192.168.2.1 in the network configuration page.
- Bioyond Works will automatically establish communication with robot

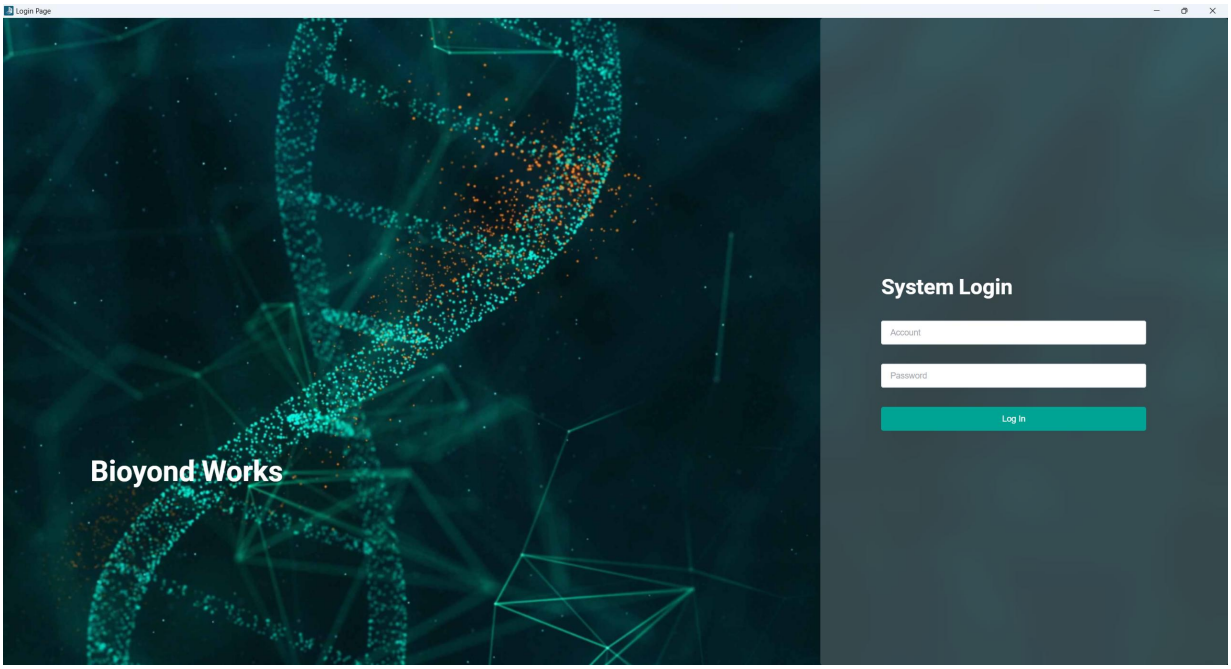
3.5. Login

After the PC connects to the workstation, double-click the Bioyond Works-S icon created on the desktop during installation to launch the software. Once launched, the user login page will be displayed.

The default login account is **admin** and password is **Admin@123** .

⚠Warning: You can change your password after logging in. Please manage your password properly to avoid losing it.

After entering your credentials, you will access the software homepage,where you can begin using the software.



3.6. Execution Program

A preconfigured robot demo process is stored within the robot controller. Upon software startup, it will automatically retrieve the programs from the robot controller. The steps for executing a program are as follows:

1) Edit and program

Open and edit the main program main.lua file in the "Program" page. You can edit the subprograms to be executed or add new motion instructions as needed.

2) Adjust motion speed

To ensure safety during the robot's motion, drag the speed slider in the Jog Control page to reduce the robot's movement speed.

3) Enable the controller

Click the "Enabled" button to set the controller to the servo-enabled state, preparing the robot to execute tasks.

4) Run the program

Click the button to start the program. Once the program starts, the button will change to . In the top-right corner of the software, the status bar will show the real-time execution status.

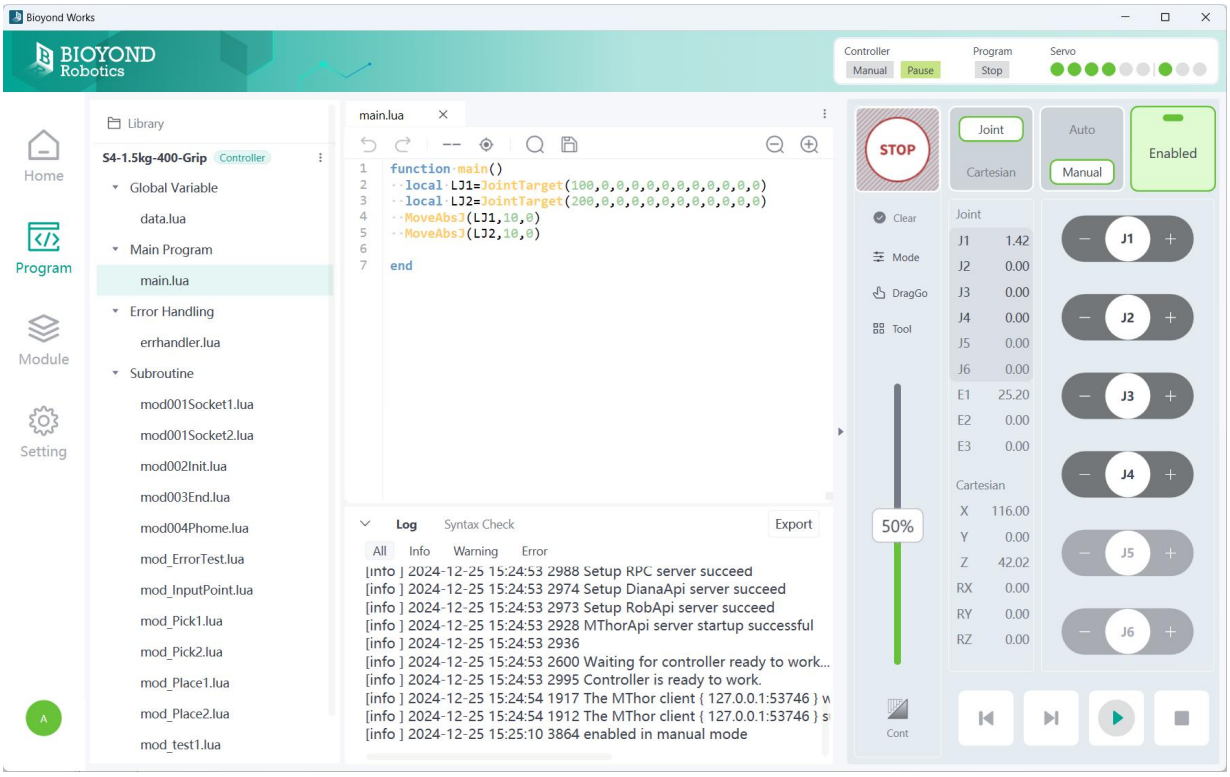
5) Pause and stop the program

Pause the program: Click the button to pause the program execution.

Stop the program: Click the button to terminate the current program.

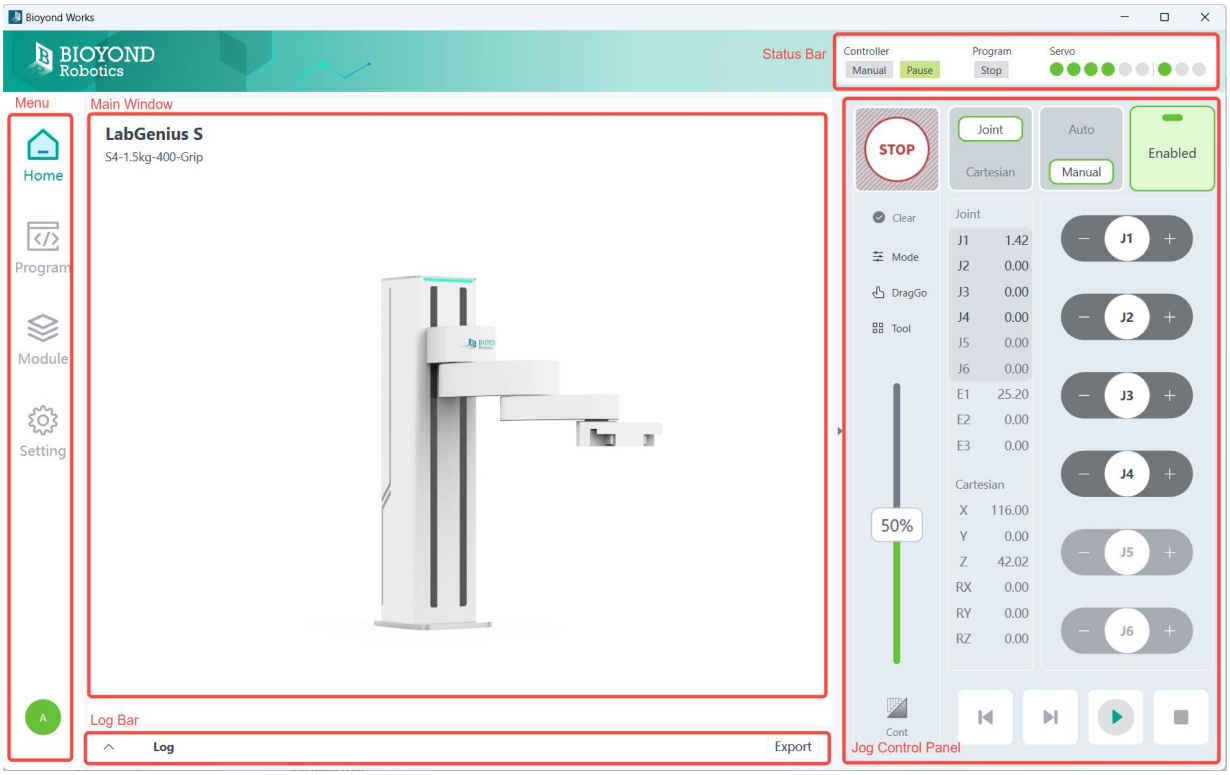
6) Emergency stop

In case of an emergency, press the emergency stop button. The system will immediately stop and disable the motor. After the emergency situation is resolved, press the emergency stop button again to restore the controller to its normal state.



4. Home Page

Functions: The Home page consists of five sections: Main Menu, Main Window, Log Bar, Status Bar, and Jog Control Panel. It displays the controller's name and image, enables log viewing and exporting, shows the current status of the controller, program, mode, and servo, and provides a manual control panel for easy operation.



4.1. Main Menu

The Main Menu allows users to navigate between the Home, Program, Module, and Setting pages.

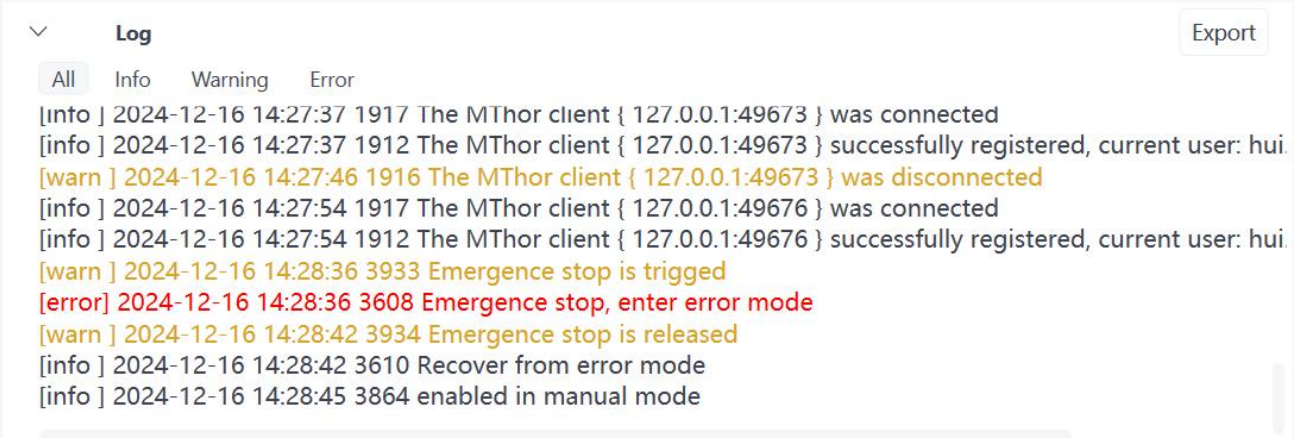
4.2. Main Window

The Main Window displays key information and functionalities specific to the current page. On the Home page, it shows the controller's name and image. The image is automatically assigned based on the robot type.

4.3. Log Bar

The log displays all operations recorded since the robot's controller was powered on.

To access it, click "^" or "**Log**" to open the log bar. Once expanded, you can resize the log bar window by hovering the mouse over its upper border.



The log information is formatted as follows: [Level] Time ID Content, where the text in [ ] indicates the log level:

- [info] Information level, in black font, provides general operational updates or status notification.
- [warn] Warning level, in yellow font, alerts users to potential issues that require caution.
- [error] Error level, in red font, signals an abnormal operation of the controller or software.

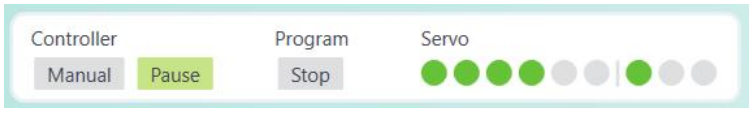
Users can filter logs by clicking "**All**", "**Info**", "**Warning**", or "**Error**". Click "**Export**" to save all current logs to a specified location on the computer.

4.4. Status Bar

In offline mode, the status bar displays as "Offline".



In online mode, from left to right, the status bar shows: Mode, Controller Status, Program Status, and Servo Status.



Mode: Displays the operating mode: Manual Mode, Automatic Mode, or Remote Mode.

Controller Status: Reflects the controller's current status: Error, Stop, Paused, Ready, Moving, or DragGo.

- Error: Indicates a system or the controller is in "Emergency Stop" status. Specific error details can be viewed in the log bar.
- Stop: Indicates the system is functioning normally, but it cannot run programs or accept motion commands (all motors are disabled in this state).
- Paused, Ready, Moving: Indicates the system is operating normally, allowing programs to run and accepting motion commands (all motors are enabled).
- DragGo: Indicates the system is normal and supports dragging motions, but it cannot run programs or accept motion commands.

Program: Three program execution states: Stop, Pause, and Running.

Servo Status: Servo Status include Normal, Enabled, and Error. Each circle represents a servo, with the first six correspond to the robot's joints, while the last three correspond to external axis status(e.g. sliding table or external rotating shaft).

- Gray: The controller is disconnected from this servo.
- Blue: The servo is functioning normally but it not enabled.
- Green: The servo is enabled, motion commands can be received and executed.
- Red: The servo has encountered an error. The system cannot run properly. Rectify the fault after troubleshooting.

5. Jog Control Panel

Functions: Emergency stop, Clear error, Mode selection, DragGo, Common tools (Home, IO control, Save Probe), Auto/Manual control, Jog control (including joint/cartesian motion, continuous/step motion).



5.1. Emergency Stop

Pressing the "Emergency Stop" button cuts off power instantly, stopping the system in an emergency. This action disables the motors, halts programs execution or block motion commands. If the motors are equipped with brakes, they will engage. The Appearance of "Emergency Stop" button will change to "Stop press to reset", as displayed on the lower right.

Once the "Emergency Stop" button is released, the controller will automatically resume operation.

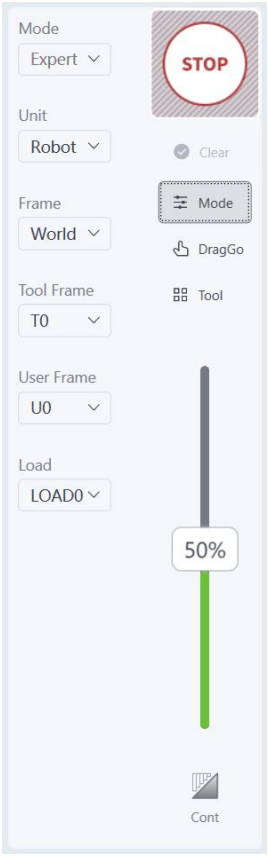




5.2. Clear Error

Under normal conditions, the "**Clear**" button is inactive.Clear Error is used to attempt to restore the system to its normal state after an anomaly occurs and troubleshooting has been performed. Clear Error is solely intended for fault recovery and cannot replace system fault diagnosis and repair.

5.3. Mode Selection



Click the "**Mode**" button to enter the Teaching Mode settings.

User Mode: Currently, only the "Expert" mode is supported.

Unit: Select either "Robot"(for controlling J1-J6 joints) or "Ext Axis"(for controlling E1-E3 joints). The effect is the same as clicking the joint or the cartesian position information area to switch the corresponding Jog button.

Frame: Select the reference coordinate system for the robot's spatial motion from the following options:

- World: Origin from the center of the robot's base.
- Base: The fixed reference point is positioned at the center of robot's base.
- User: Using the user coordinate system selected in the dropdown during teaching.
- Tool: Using the tool coordinate system selected in the dropdown during teaching.

Note on teaching reference coordinate systems: In spatial motion, when using the axis operation keys X, Y and Z, the robot TCP moves along the corresponding x, y and z axes of the selected reference coordinate system. When using the axis operation keys Rx, Ry and Rz, the robot rotates around the x, y and z axes of the reference coordinate system while TCP position is fixed.

Tool Frame: Define in "Program - Global Variables - data.lua" or added and calibrated in "Setting - Coordinate and Load - Tool Frame".

User Frame: Define in "Program - Global Variables - data.lua" or added and calibrated in "Setting - Coordinate and Load - User Frame".

Load: Define in "Program - Global Variables - data.lua" or added in "Setting - Coordinate and Load - Load".

5.4. DragGo

DragGo function allows operators to manually move the end of the robotic arm to position it at a certain location or execute specific actions, making the teaching process more intuitive and easier to understand.

DragGo mode only works in manual mode and is not available when the system is in auto mode. First, enter the DragGo mode when system is disabled. Then click "**Enable**" to make the DragGo mode take effect, that is, the correct sequence for activating DragGo Mode is: "**Manual**" → "**DragGo**" → "**Enable**".

⚠ Warning: In DragGo mode, ensure the safety of the robot and the surrounding environment to avoid collision or damage caused by dragging.

Once the system is in DragGo Status, you can move the robot to the desired position, add Move class instructions in the program editor. The default coordinates will reflect the robot's current position and posture.

DragGo can be stopped directly if it is not enabled; If system is enabled, the servo must first be disabled before clicking "**DragGo**" button to disable dragging.

DragGo parameters can be adjusted in "Settings - System Setting - DragGo". A higher adjustment coefficient (range from -1 to 1) makes dragging easier but increases inertia when stopping.

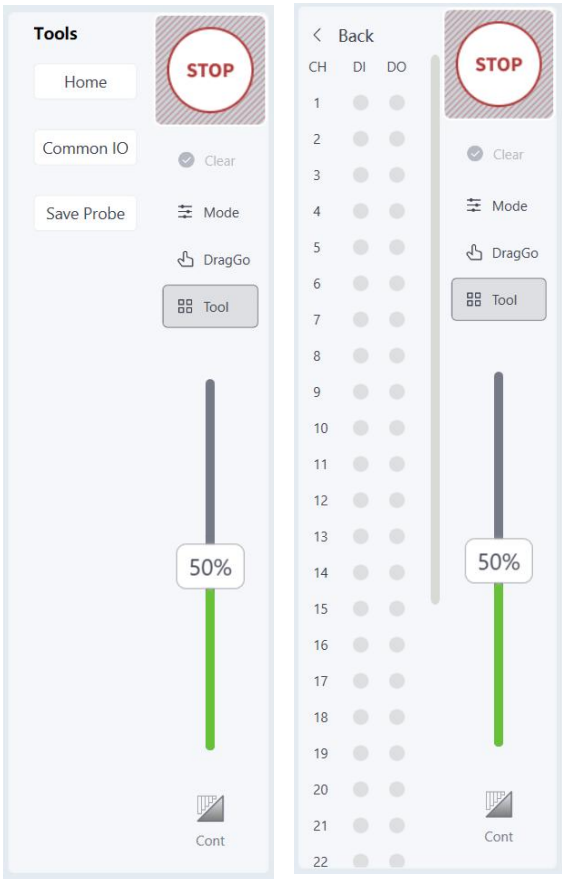
5.5. Common Tools

Click the "**Tool**" button to access common tools: Home, Common IO, Save Path.

Home: Clicking the "**Home**" button move all robot joints back to the user-defined home position. The user home position can be set in "Setting - System Setting - User Home".

Common IO: This feature displays the input and output signal status for User IO pins 1 to 64, Virtual IO pins 1 to 32, and System IO pins 1 to 16. Each pin's indicator light reflects the signal status: 0 (gray) or 1 (green).

Save Probe: Exports the current robot motion trajectory to a designated location on the computer. The operation information of the robot within 4 minutes before the export time is recorded, which is often used to monitor and diagnose the robot performance.



5.6. Auto/Manual Control

The "Auto/Manual" button contains two operation modes for controlling the robot:

- Manual mode(Default)
  Speed Limitations: the maximum speed of the robot is limited to 20deg/s for joint motion and 200mm/s for linear motion.
- Auto mode(Default)
  Click the "Auto/Manuall" button to toggle to Auto Mode, the robot operates at its full capability.

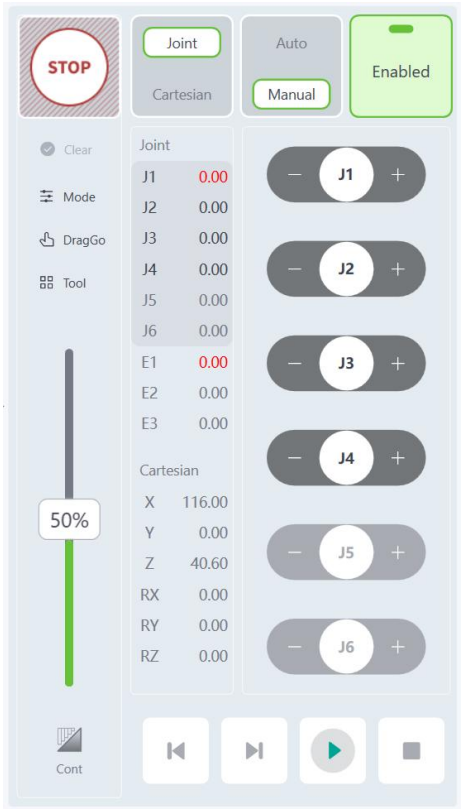
5.7. Jog Control

1) Joint/Cartesian

The Jog Control Panel displays joint information for both the robot's joints (J1 to J6) and external axes(E1 to E3), with rotational axes measured in degrees (deg) and linear axes measured in millimeters (mm). It also shows the robot pose in cartesian space based on the tool and user coordinate system in the selected "Mode".

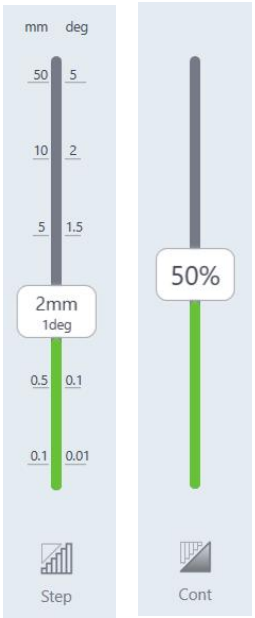
Users can click on the joint information area or the cartesian position area to switch the control mode of the Jog buttons.

- Joint information: The Jog buttons then control each of robot's joints (J1 to J6) and the external axes (E1 to E3) individually.
- Cartesian position area: The Jog buttons adjust the robot's linear positions (X, Y, Z) and rotation angle (Rx, Ry, Rz)





2) Continuous/Step



In the Jog Control Panel, users can switch between the speed and increment sliders by clicking the "**Cont**" or "**Step**" buttons, allowing value adjustments. The increment amount on the left side of the slider is for linear axis joints, measured in millimeters (mm). The increment amount on the right side of the slider is for rotational axis joints, measured in degrees (deg).

- In auto mode, only the speed slider in continuous mode is displayed, and switching to "Step" is not permitted. The settings of the speed slider are applied during program execution.
- In Manual Mode, users have access to both the speed and increment sliders, enabling flexible adjustments of the robot's motion speed and increment amount based on specific needs.

3) Jog control

The Jog buttons control the movement of the robot and external axes, functioning only in manual mode when the robot is enabled. There are two ways to activate the Jog buttons:

- Tap or long-press the "—" or "+" buttons.
- Slide the joystick button to the left or right.

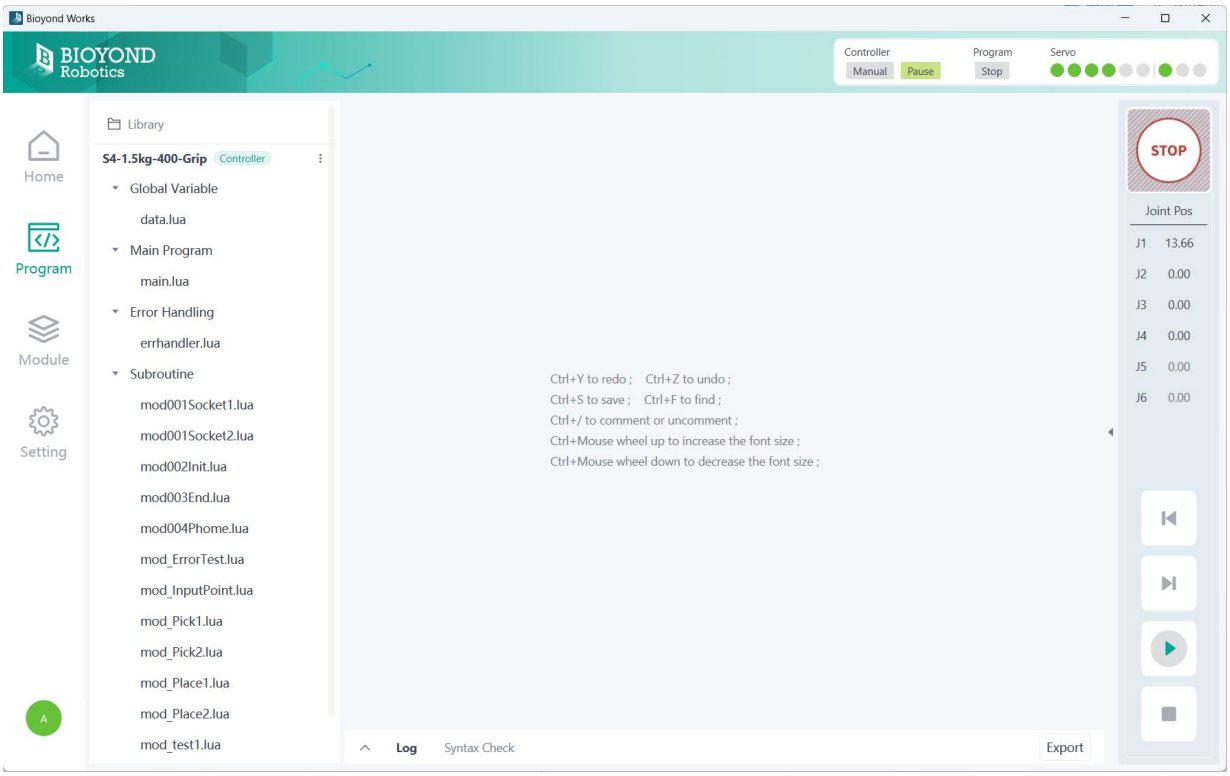
In step Mode, the Jog buttons can only be activated once per tap or joystick movement. After each activation, the button must be released, or the joystick must return to its original position before the next operation. In this mode, the Jog buttons are influenced solely by the increment amount, with adjustments to the increment slider directly affecting the displacement for each jog.

In Continuous Mode, the Jog buttons control movement via tapping, long-pressing, or joystick operation. Movement speed is governed solely by the speed slider, while the increment setting is inactive in this mode.

6. Program

Functions: The Program section provides a comprehensive set of features for managing projects and subroutines:

- **Project management:** Includes publishing, importing, exporting, renaming, deleting, and creating projects.
- **Subroutine management:** Allows creating, saving, importing, exporting, renaming, and deleting subroutines.
- **Coding assistance:** Provide features like code completion, add motion instructions to get the current pose, update pose with move instruction, error checking, and shortcut keys.
- **Program execution:** Debug, run, pause, stop programs, and shows the run pointer.



6.1. Program Library

The file tree shows the structure of projects and their subroutines. A maximum of two project types can be displayed:

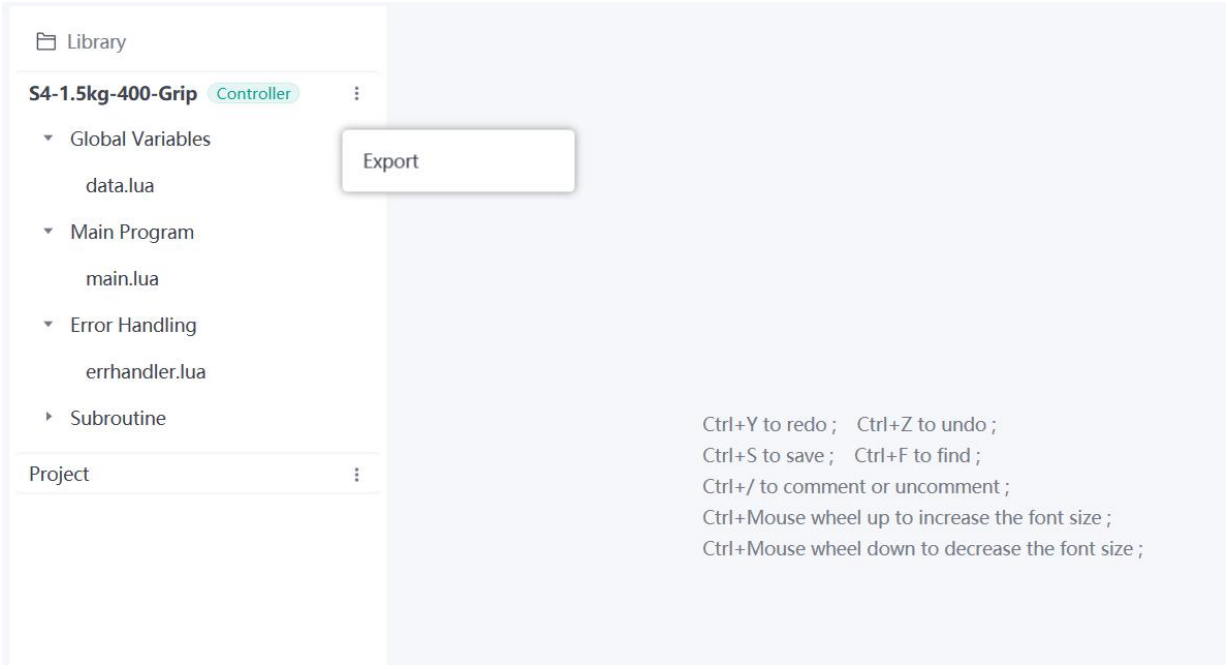
Online projects: Projects stored in the robot controller. When the computer is connected to the controller, it automatically retrieves the current projects.

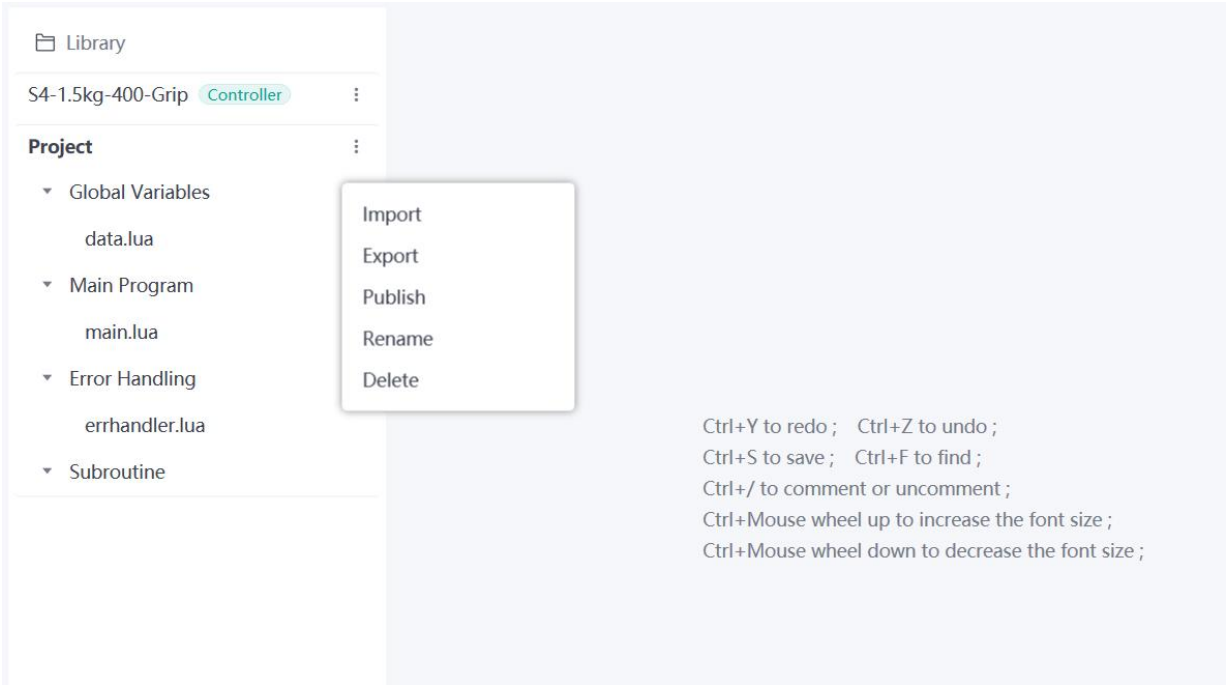
Offline projects: Projects that have been edited offline and are not yet synced with the controller.

Each project consists of the following components: a single set of global variables, a single main program, a single error handling program, and their subroutines. Unlimited number of subroutines can be included to support tasks or modularize projects.

6.2. Project Management

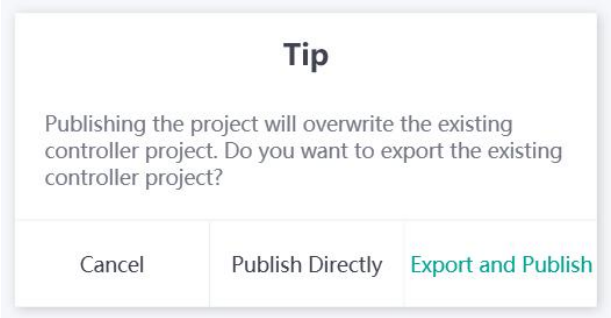
For online projects, available operations is exporting.For offline projects, operations includes publishing, importing, exporting, renaming, deleting, and creating new projects.





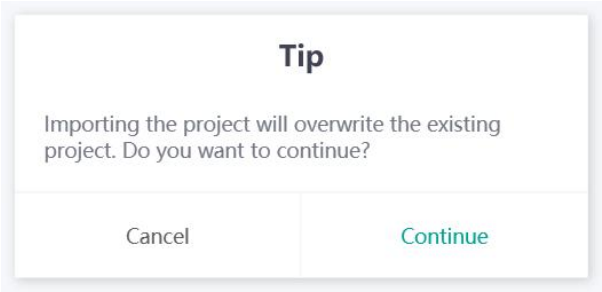
1) Publish

This operation publishes the offline project to the controller, overwriting the existing project. A prompt will appear, offering the following options: cancel, publish directly, or export and publish (saves the current project in the controller before publishing the offline project). If modifications are made to the online project and no further modifications within 1 seconds, the project is automatically published and syntax-checked.



2) Import

This operation imports projects from the external storage of the computer into the offline project. Importing will overwrite the existing offline project, with a prompt appearing to confirm this action.



3) Export

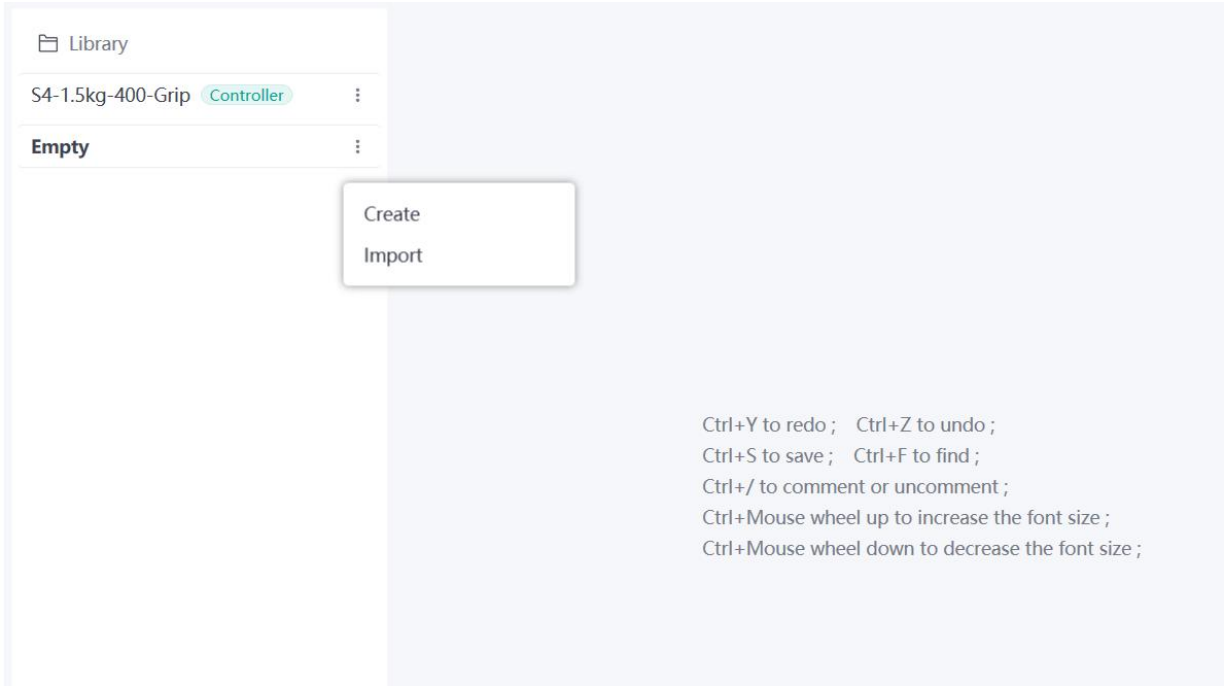
This operation exports online or offline projects to a specified location in the computer's external storage.

4) Rename

This operation allows renaming of offline projects, while renaming online projects is not allowed.The name of the offline project supports Chinese, English, and numbers, but does not support underscores and punctuation marks.

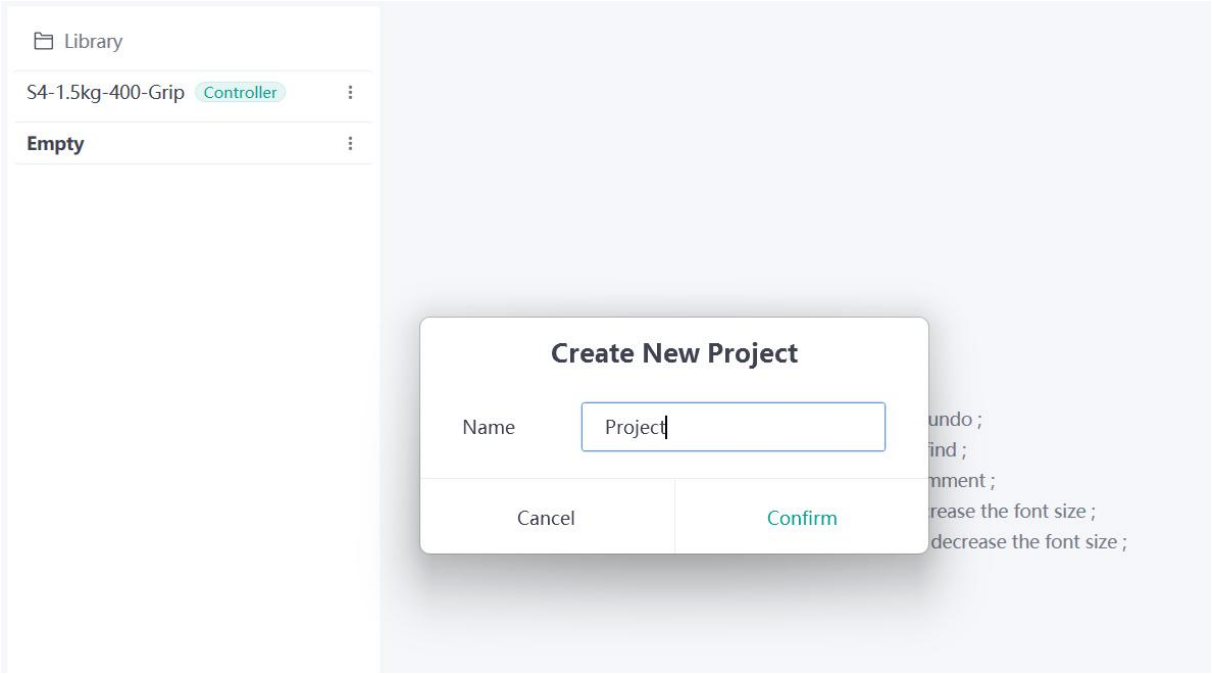
5) Delete

This operation deletes offline projects from the software. Once deleted, the offline project becomes an "Empty", ready for creating or importing new offline projects.



6) Create

The name of a newly created offline project supports Chinese, English, and numbers, but does not support underscores and punctuation marks. This operation creates a new offline project automatically generates a framework includes global variables, main program, error handling, and subroutines.

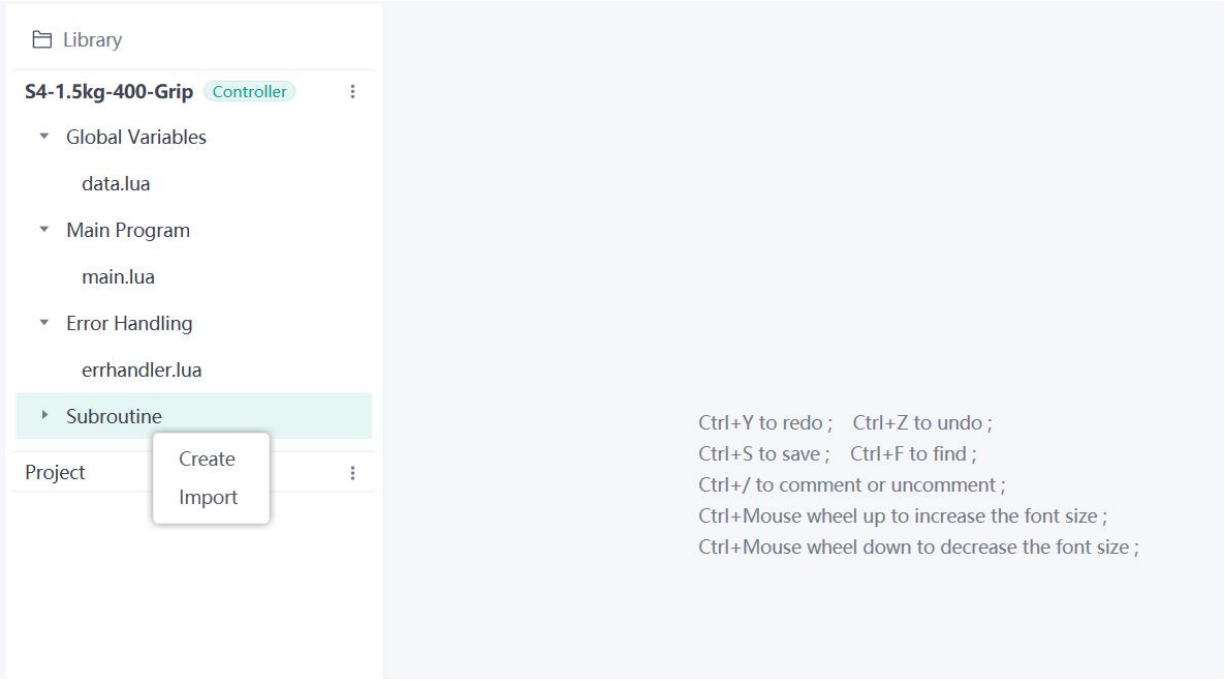


6.3. Subroutine Management

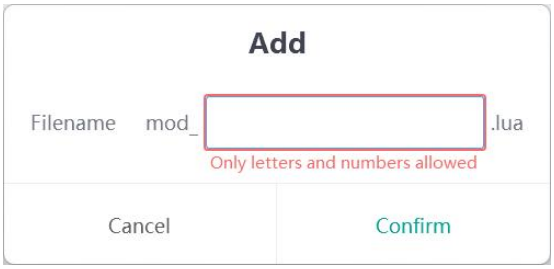
Subroutine operations include creat, save, import, export, rename and delete.

1) Create

Right-click on "**Subroutine**" to create a new program under it.



Because subroutines will be called in the main program, the name of the subroutines only support letters and numbers.

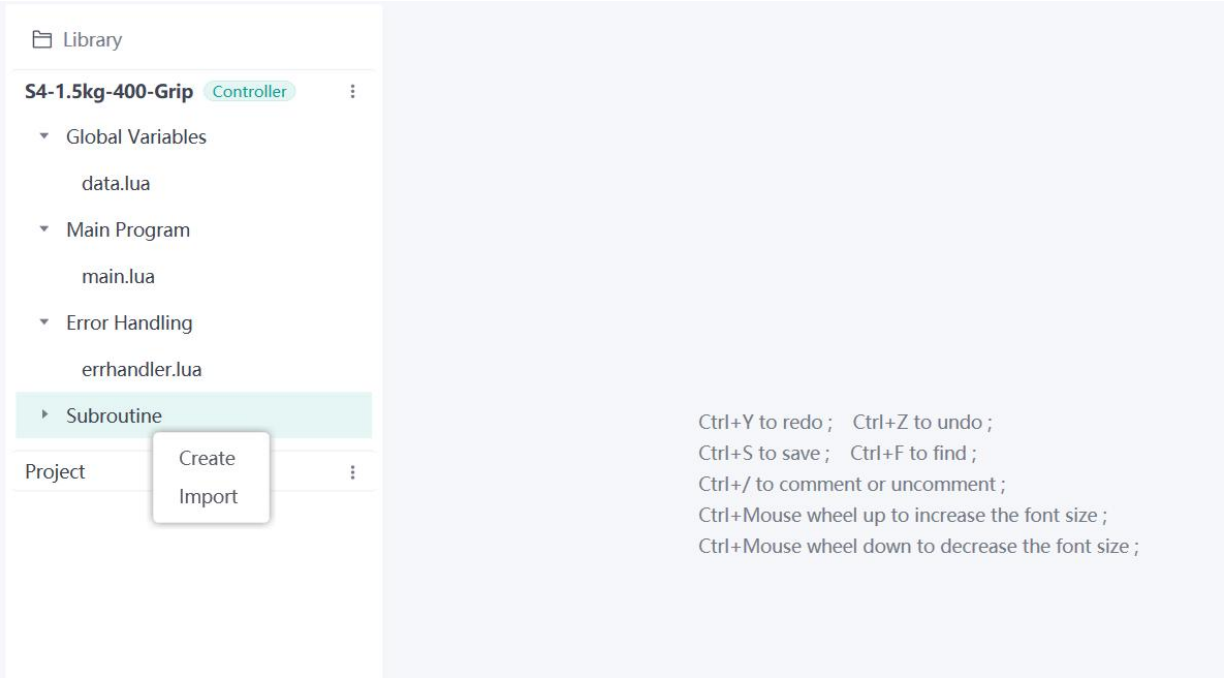


2) Save

For online project, modifications are automatically saved and published. For offline projects, the user must save changes by using the shortcut key “Ctrl+S” or clicking the button icon "".

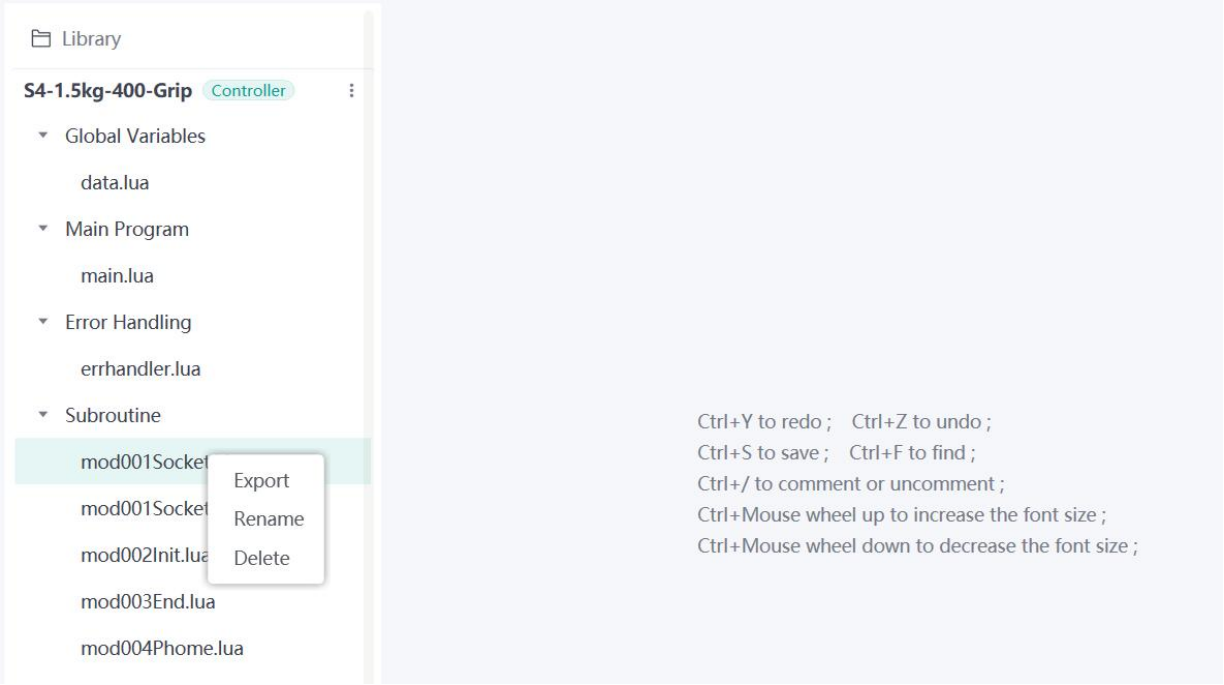
3) Import

Right-click on "**Subroutine**" to import the program into the Subroutine.



4) Export

Select a subroutine, right-click and click "**Export**" to export the currently selected subroutine to a specified location.



5) Rename

Select a subroutine, right-click and click "**Rename**" to modify the name of the currently selected subroutine.

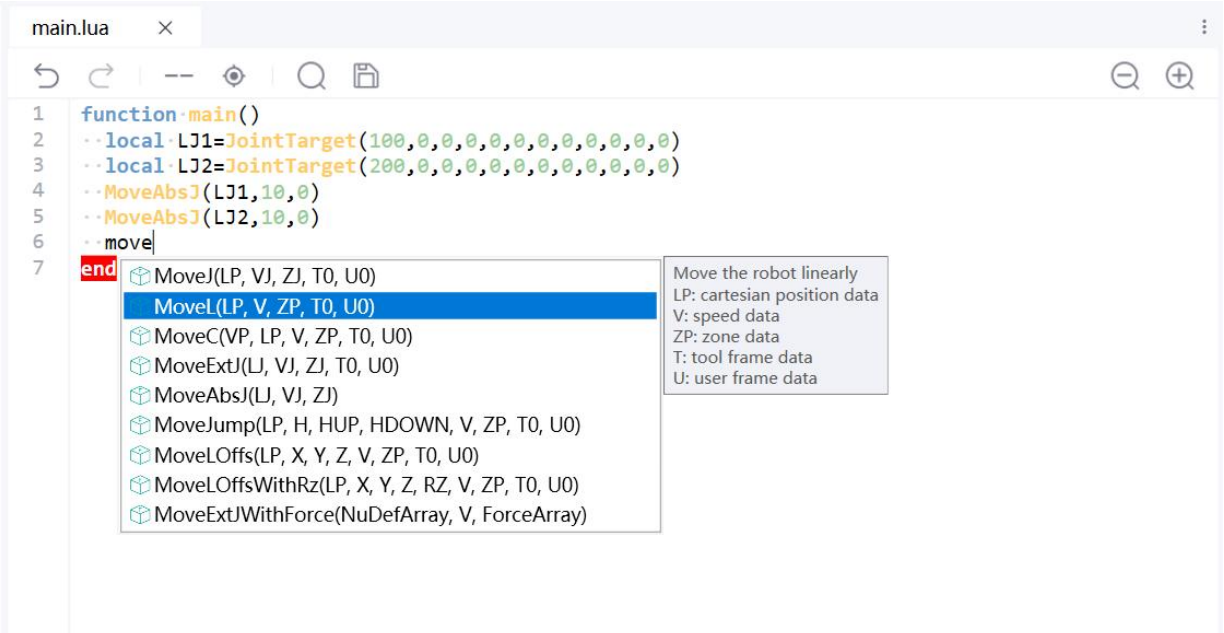
6) Delete

Select a subroutine, right-click and click "**Delete**" to delete the currently selected subroutine.

6.4. Coding Assistance

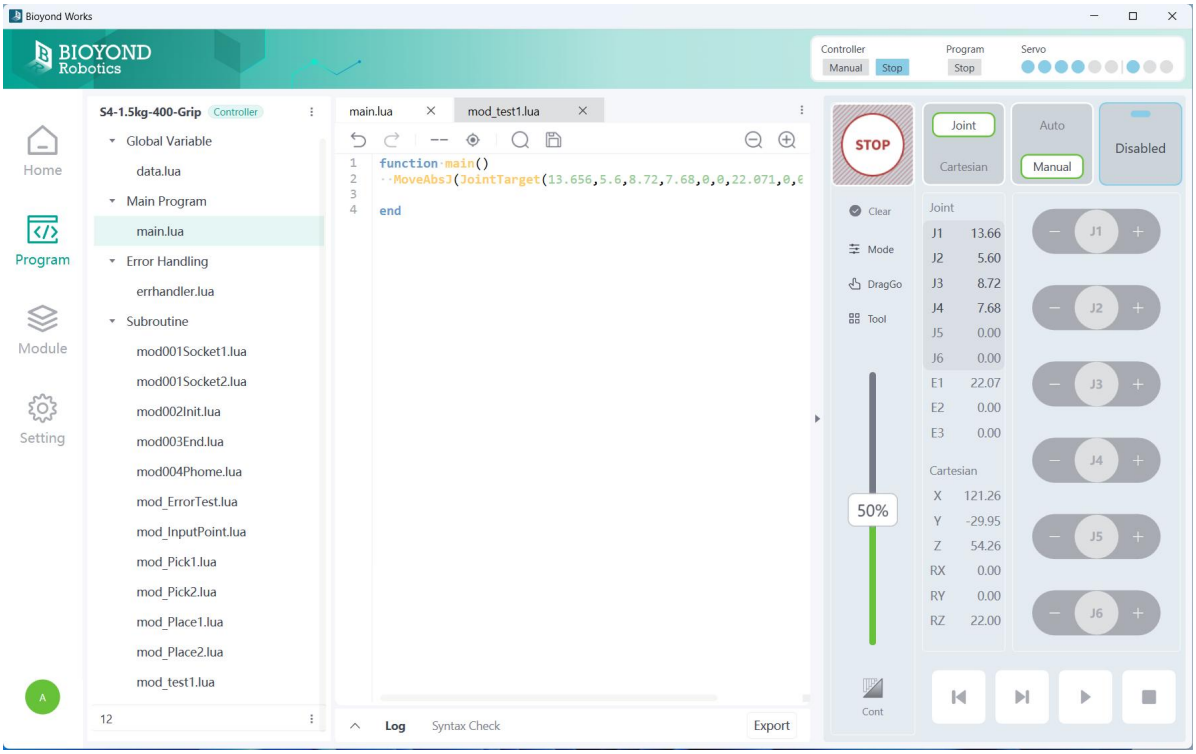
1) Code completion

The program editor provides automated instructions for completing the current line of code coding. Detailed introductions to specific commands are provided in "Appendix A: Commands and Data Types".



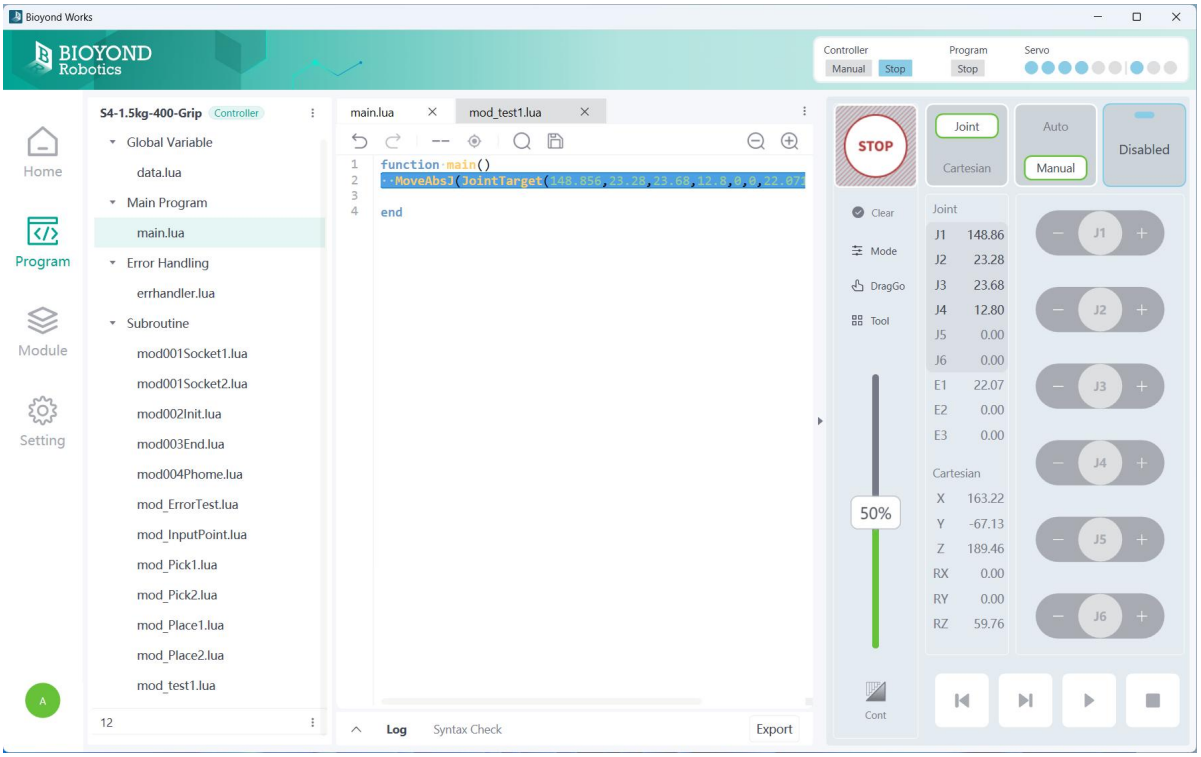
2) Add motion instructions to get the current pose

After adding motion instructions within the Lua script editor, you can see that the default coordinates are the current pose of the robot. With this function, you can first jog the robot to the desired position, and then add motion instructions within the Lua script editor, facilitating robot teaching.



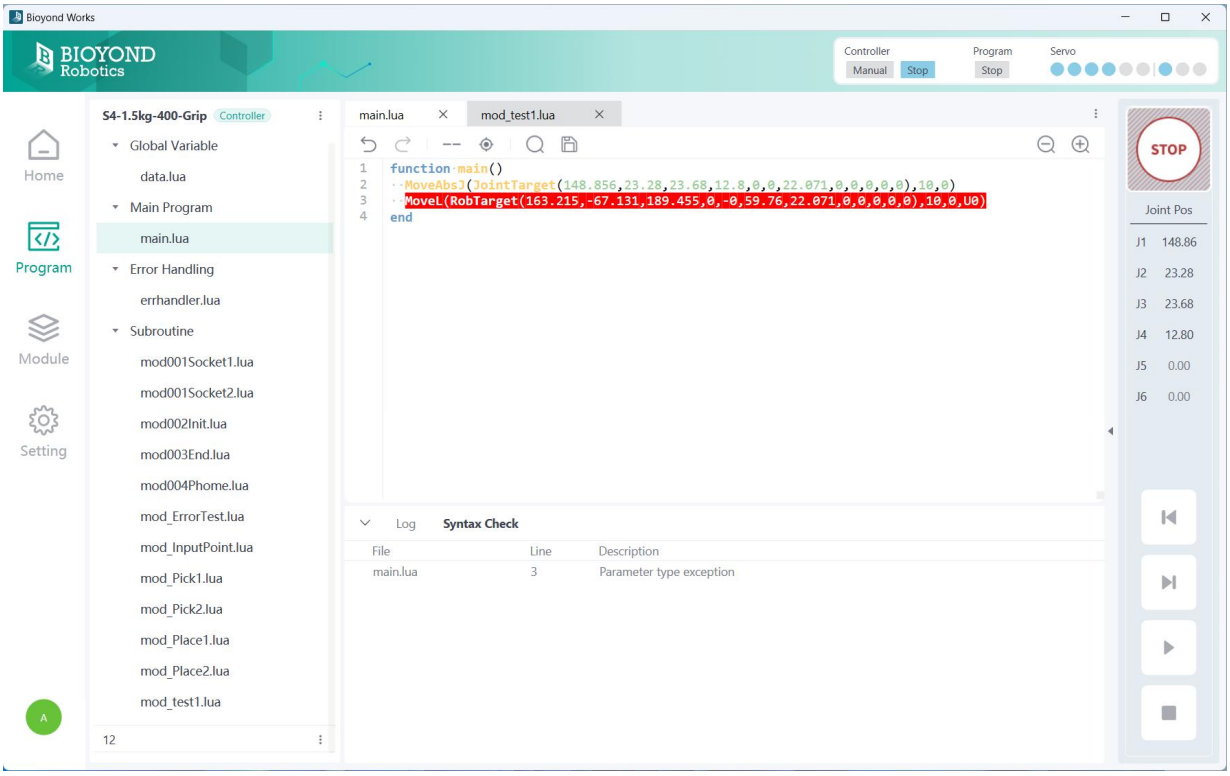
3) Update pose with move instruction

Place the mouse cursor on the motion instructions and click "⊙"in the toolbar to get the robot current pose. You can see that the target point pose of the motion instructions has been updated to the current pose of the robot.



4) Syntax checking

The program editor automatically checks code for correct syntax. A message is displayed when users enter code that contains a syntax error, and the suspect code is highlighted.
Double-click a line of information in the "Syntax Check" list to open the subroutine with the error.



6.5. Shortcut key

The shortcut keys for the program editor are shown alongside their corresponding icon buttons. The "-" indicates that no shortcut key or icon button is assigned to that function:

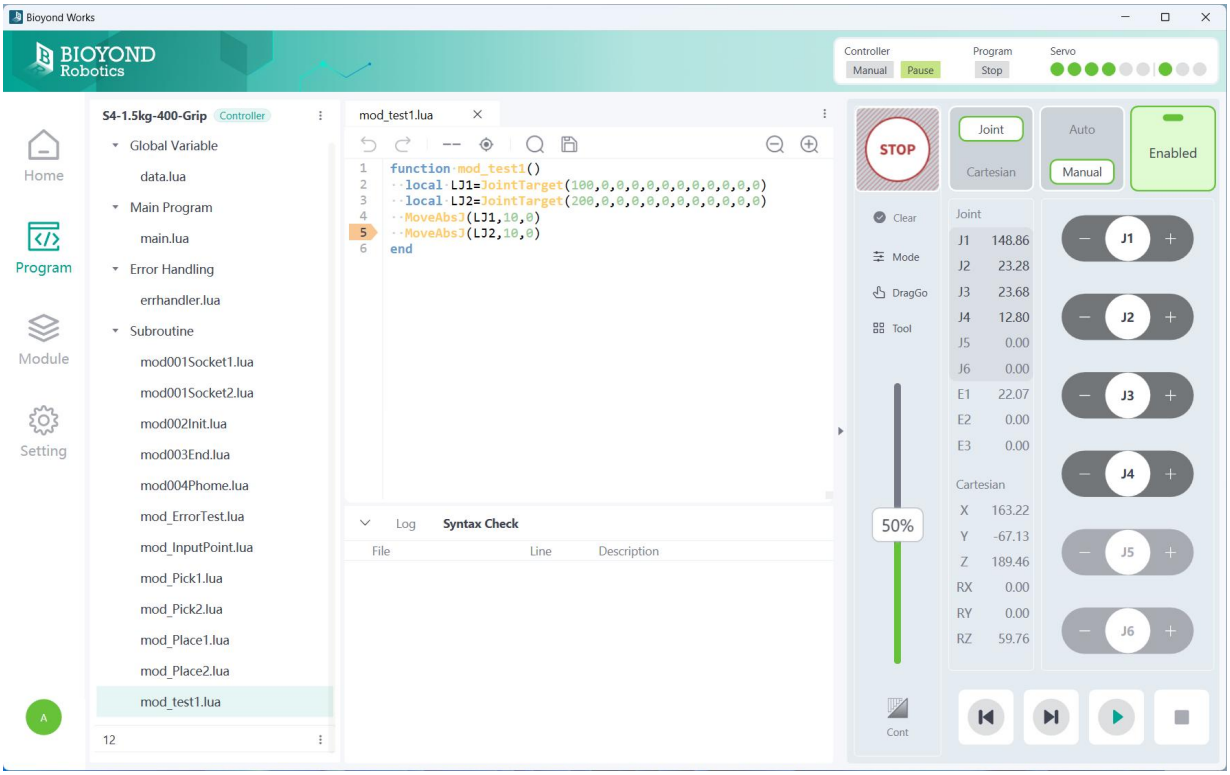
Purpose	Shortcut Key	Icon
Save	Ctrl+S	
Redo	Ctrl+Y	
Undo	Ctrl+Z	
Find	Ctrl+F	
Comment or uncomment	Ctrl+/	
Increase font size	Ctrl+Mouse wheel up	
Decrease font size	Ctrl+Mouse wheel down	
Update pose with move instruction	-	

6.6. Program Execution

1) Motion instruction debugging

Debugging can only be performed in manual mode, with the system enabled, and when the program is in a stopped state. The instructions supported for debugging include MoveL, MoveJ, MoveC, MoveAbsJ, and MoveExtJ.

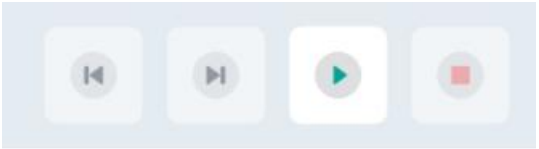
To debug, select a debug breakpoint before the motion instruction that requires debugging, the breakpoint will be indicated by an orange marker. (As shown in the image below, the breakpoint is located in front of the fifth line of code.) Then, click " " to run the line preceding the debug breakpoint, or click " " to run the line where the debug breakpoint is located. This will execute the current motion instruction up to the target position.



⚠Warning: If the program is modified during the debugging process, the currently executing motion instruction will still complete, but the debug breakpoint will be automatically canceled. After switching to another debug breakpoint, debugging of other motion instructions can continue.

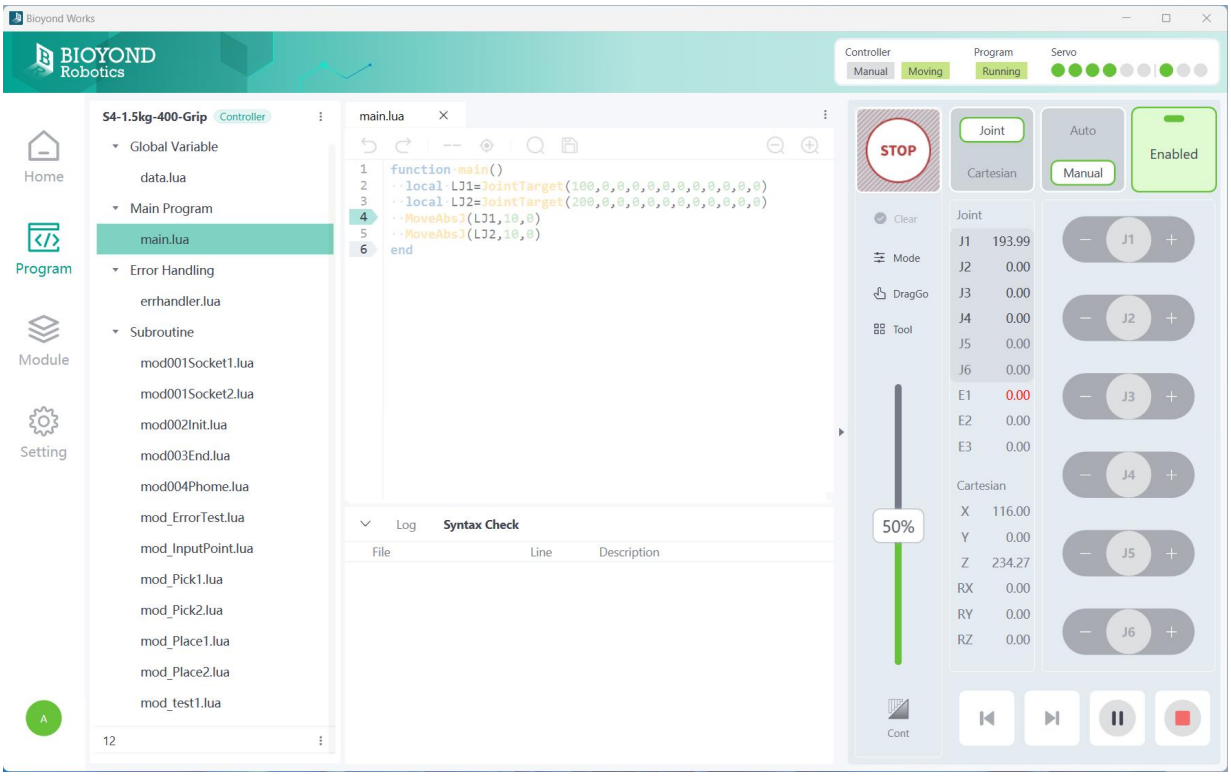
2) Run program

Click the " " button to run the program when the servo is enabled. Once the program starts, the button icon changes to " ". At this point, clicking the " " button pauses the program, while clicking the " " button stops it.



3) Run pointer

The run pointers include the program pointer and the robot pointer. The program pointer indicates the instructions currently being processed by the controller, which is indicated by the gray breakpoint in the image below. The robot pointer indicates the motion instruction currently being executed by the robot, it is marked by a green breakpoint in the program file(as shown in the image below) .



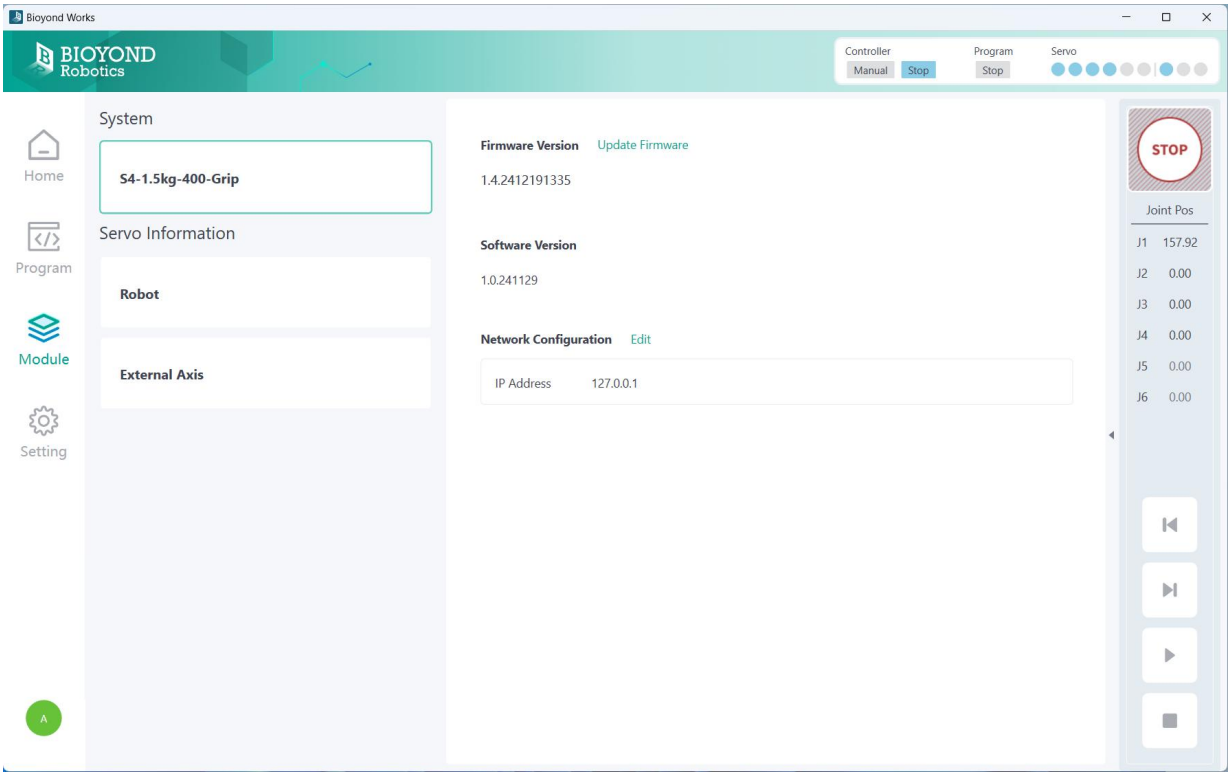
⚠ Warning: Programs are configured to run in either manual or auto modes. Before running a program, prioritize safety by verifying that the surrounding environment is free of obstacles to prevent potential accidents.

7. Module

Functions: Provides fundamental system information and operations, including:

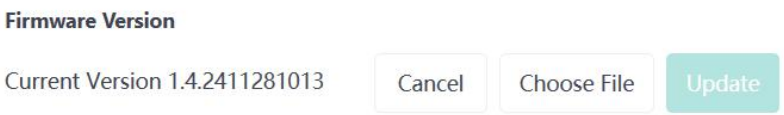
- **Basic system information:** Shows controller name, firmware version and software version.
- **Robot joint information:** Shows the status and parameters of each joint's servo motor.
- **External axis information:** Shows the status and parameters of the external axis servo motors.
- **Update firmware**
- **Configure network**

This feature helps users quickly access the basic configuration and version status of the system, facilitating necessary maintenance and upgrades.



7.1. Update Firmware

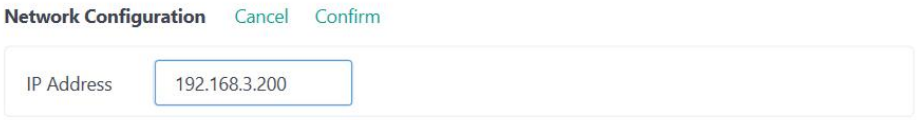
Updating the firmware is updating the robot controller and can only be done when the system is not enabled. Choose a local file with the .BioFirmware extension which contains the firmware upgrade for the robot controller, and then click the "Update" button to start the firmware upgrade process. Once the firmware update is successful, restart the controller for the changes to take effect.



7.2. Configure Network

In network configuration, the controller IP address and client IP address can be changed simultaneously. You can change the IP address only when the system is disabled. After the change takes effect, restart the controller.

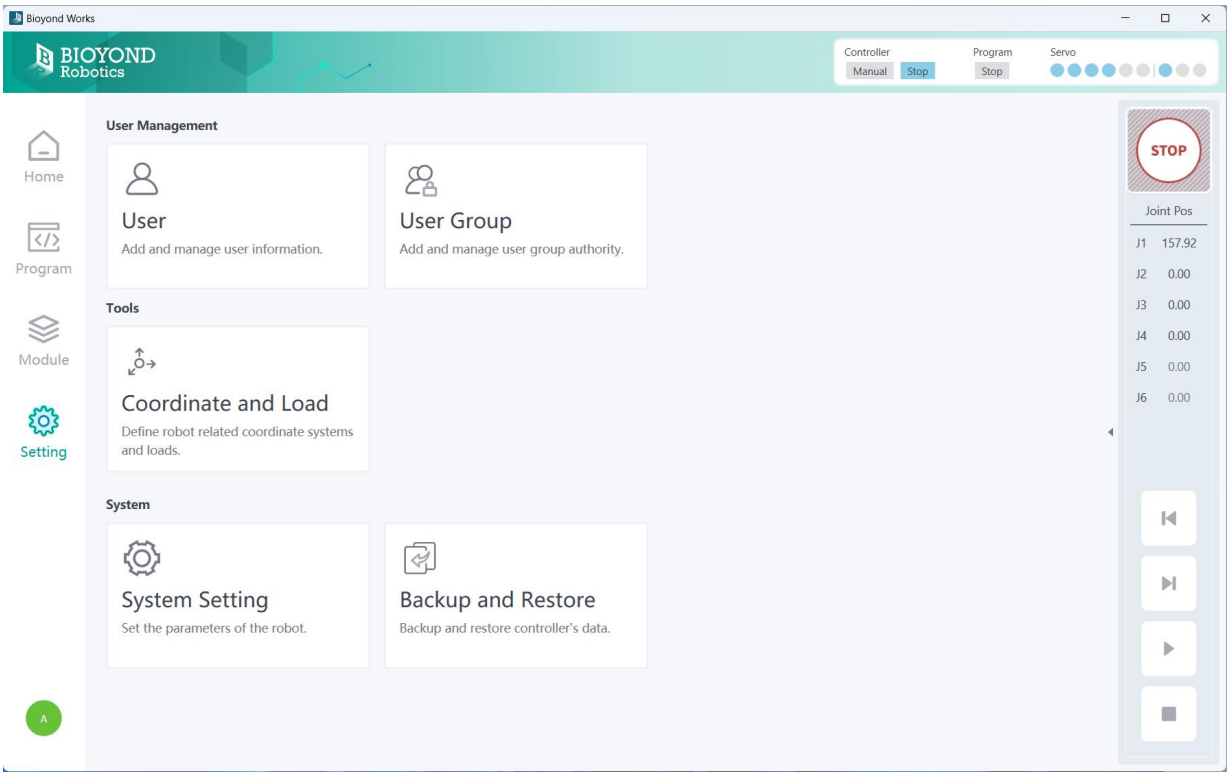
⚠Warning: This operation will change the IP of the controller, please make sure that the IP is known to all relevant users of the robot, otherwise others may not be able to connect to the robot using Bioyond Works after modification.



8. Setting

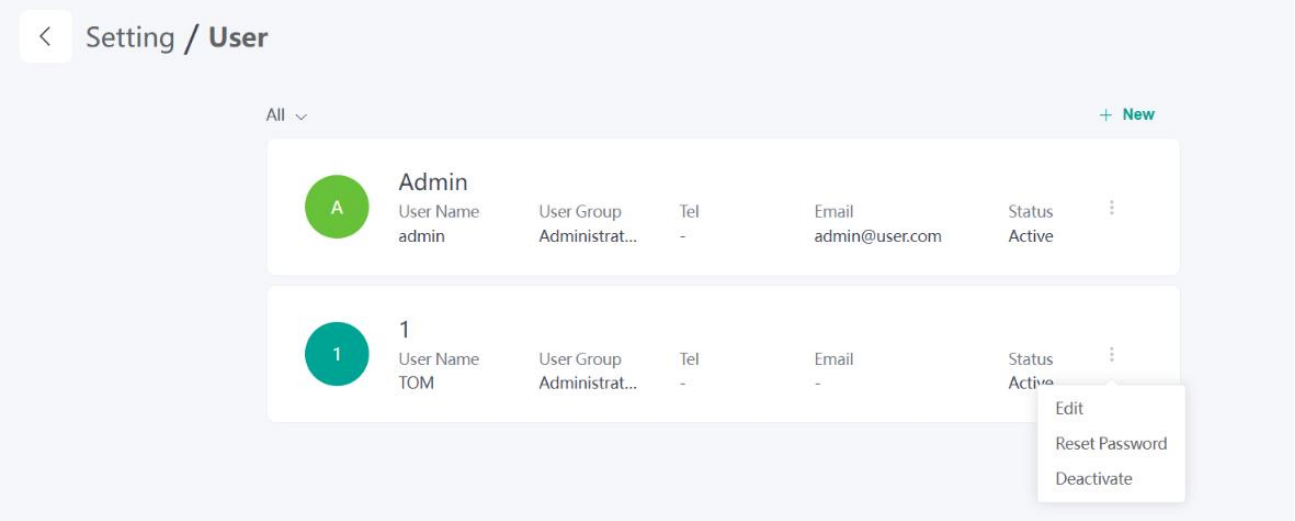
Functions: Provides options for configuring and calibrating key elements, including:

- **User management:** Manage users and user groups.
- **Coordinate and load:** Configure and calibrate tool frame, user frame and the payload parameters for optimal performance.
- **System setting:** Adjust settings for the robot and external axes, including user home, zero joint, collision detection and GragGo parameters.
- **Backup and restore:** Backup and restore controller's data.



8.1. User

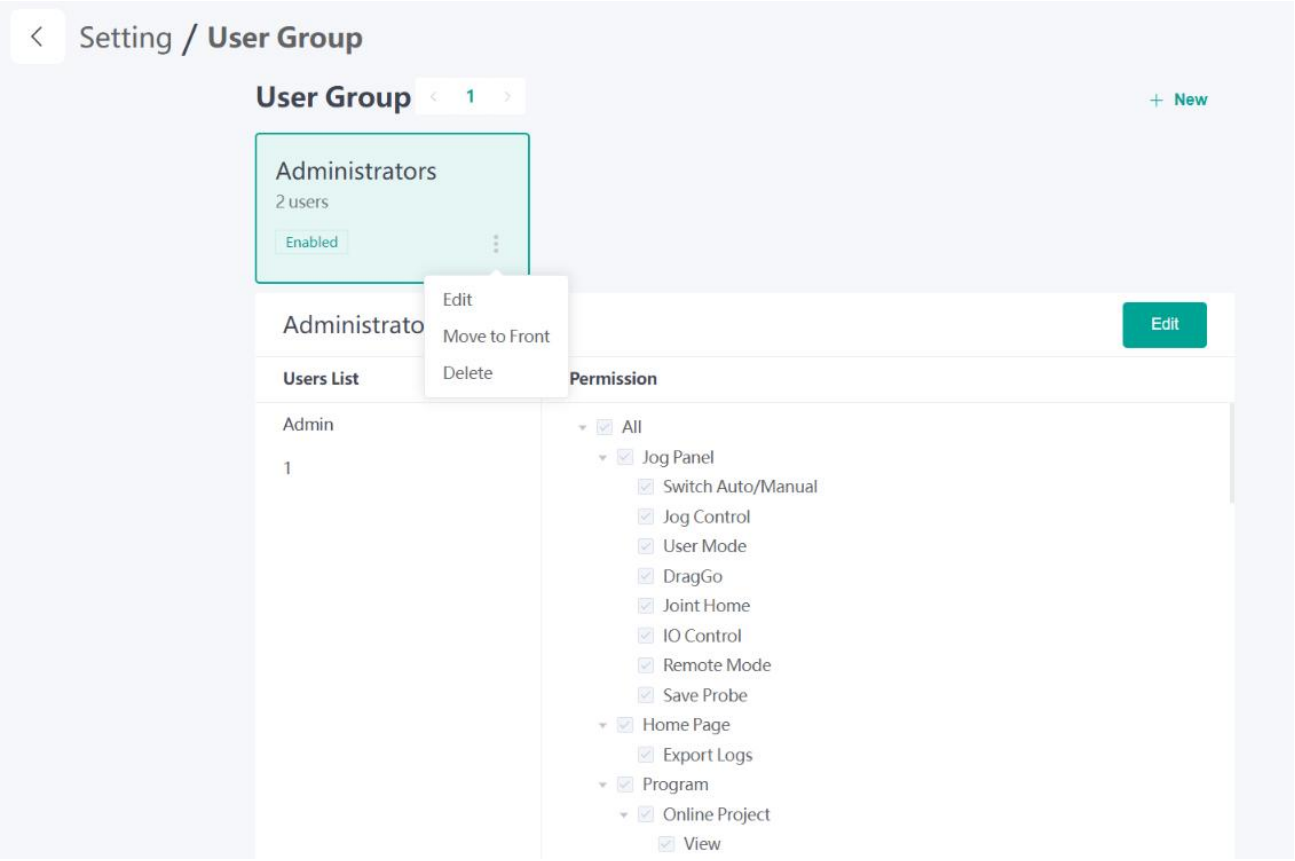
In the user list, users can be created, edited, reset their passwords, and actived/deactivated.



New user: The newly created user can log in to the system.
Edit user: Edit user information (name, user group, profile color, phone number, and email address).
Reset password: Reset the password of the user. The user uses the reset password to log in to the system. The old password is invalid.
Active/deactivate user: After the user is deactivated, the user cannot log in to the system, but can log in to the system after the user is actived.

8.2. User Group

In the user group list, you can create, edit, and delete user groups, active user groups, and configure permissions for user groups.



New user group: A newly created user group is enabled by default. An activated user group can be selected when creating and editing users (user operable permissions inherit from the assigned user group).
Edit user group: Edit the name of the user group.
Move to front: Move the current user group to the first position in the list.
Delete user group: After deleting a user group, users under that group will be unable to log in to the system.
Activate/deactivate user group: After deactivation, users under the user group will be unable to log in to the system, but can log in after the user group is activated.

8.3. Coordinate and Load

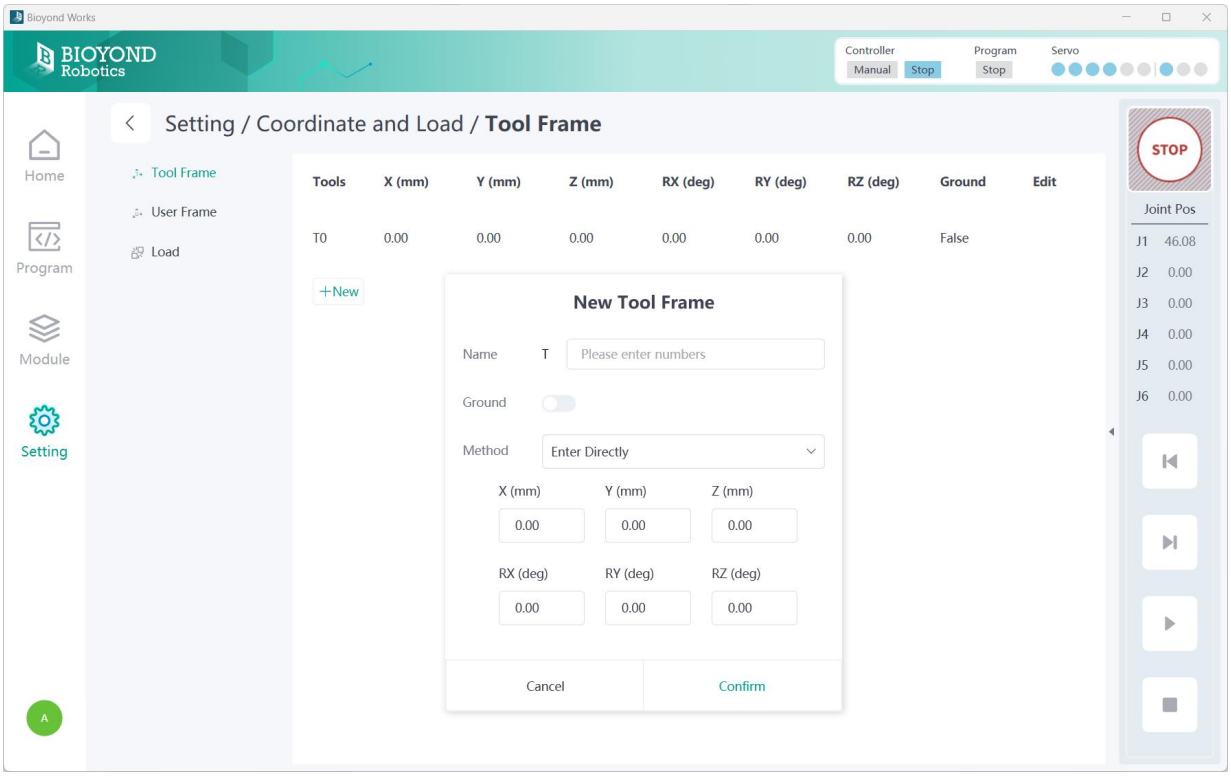
The coordinate system and payload settings can be configured when the program is stopped. Once successfully set, they will be synchronized to the "Program - Global Variables - data.lua" and the "Mode" in the Jog control panel.
Similarly, coordinates or payloads correctly defined directly in the "Program - Global Variables - data.lua" using the commands DefTool, DefUser, DefLoad (e.g., T2 = DefTool(1,2,3,4,5,6,7)) will also be synchronized to the "Coordinate and Load" page.
⚠ Warning: Incorrect definitions in data.lua will not be synchronized to the "Coordinate and Load" page, such as T2A = DefTool(1,2,3,4,5,6,7) or T2 = DefTool(2,3,4,5,6,7) with missing parameters.

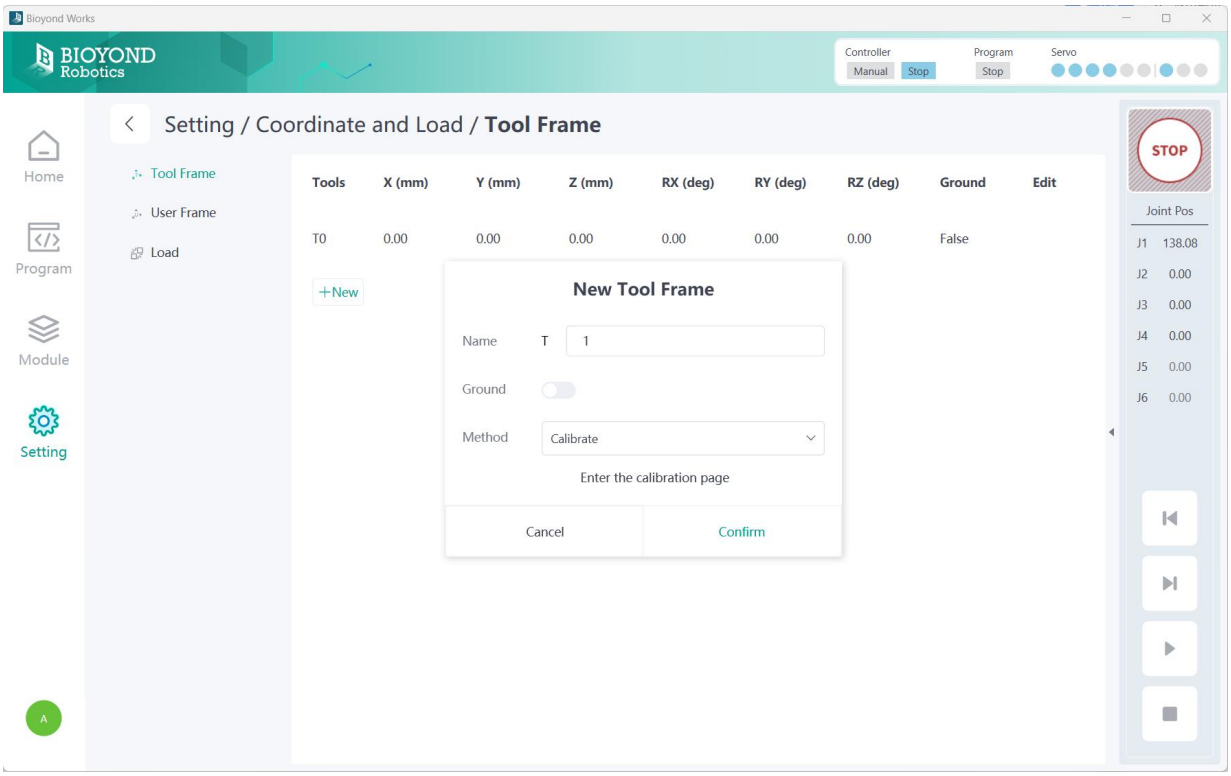
1) Tool frame

The tool frame is established with the tool center point as the origin. When changing the end tool of the robot, only the tool frame needs to be recalibrated, and the points already taught in the programming page will remain valid. The relevant operations are as follows:

■ New tool frame

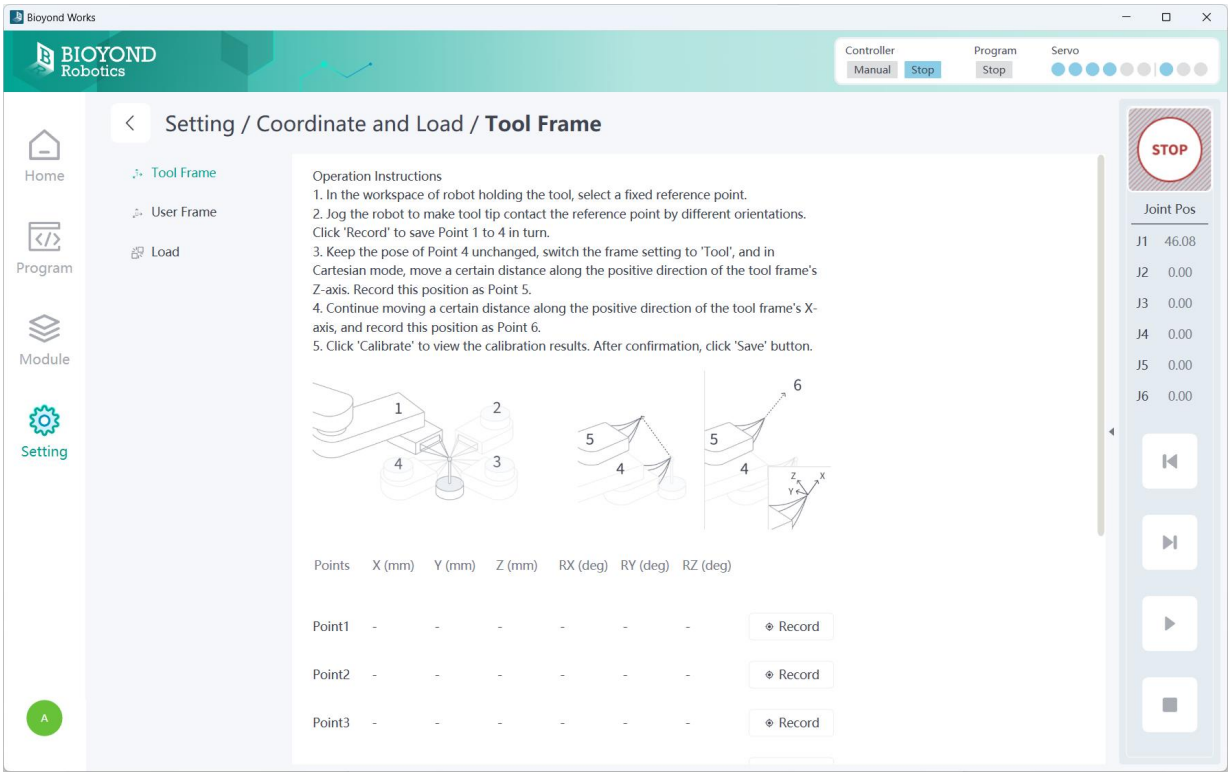
Click the " +New " button to bring up the new tool frame window. Enter the name of the tool frame, which only supports numerical characters. Currently, the tool frame is not fixed to the ground by default. Within the method, you can choose "**Direct Input**" or "**Calibrate**". If "**Calibrate**" is selected, you will proceed to the calibration page for further settings.





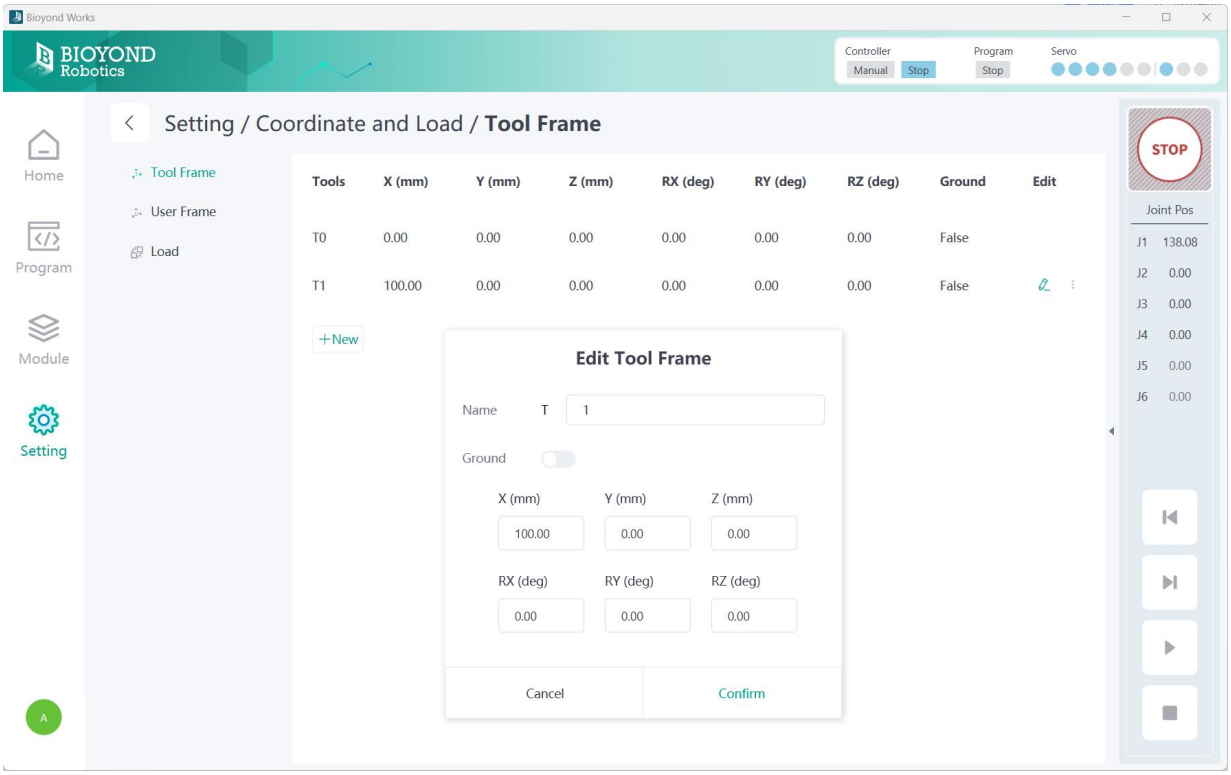
■ Calibrate

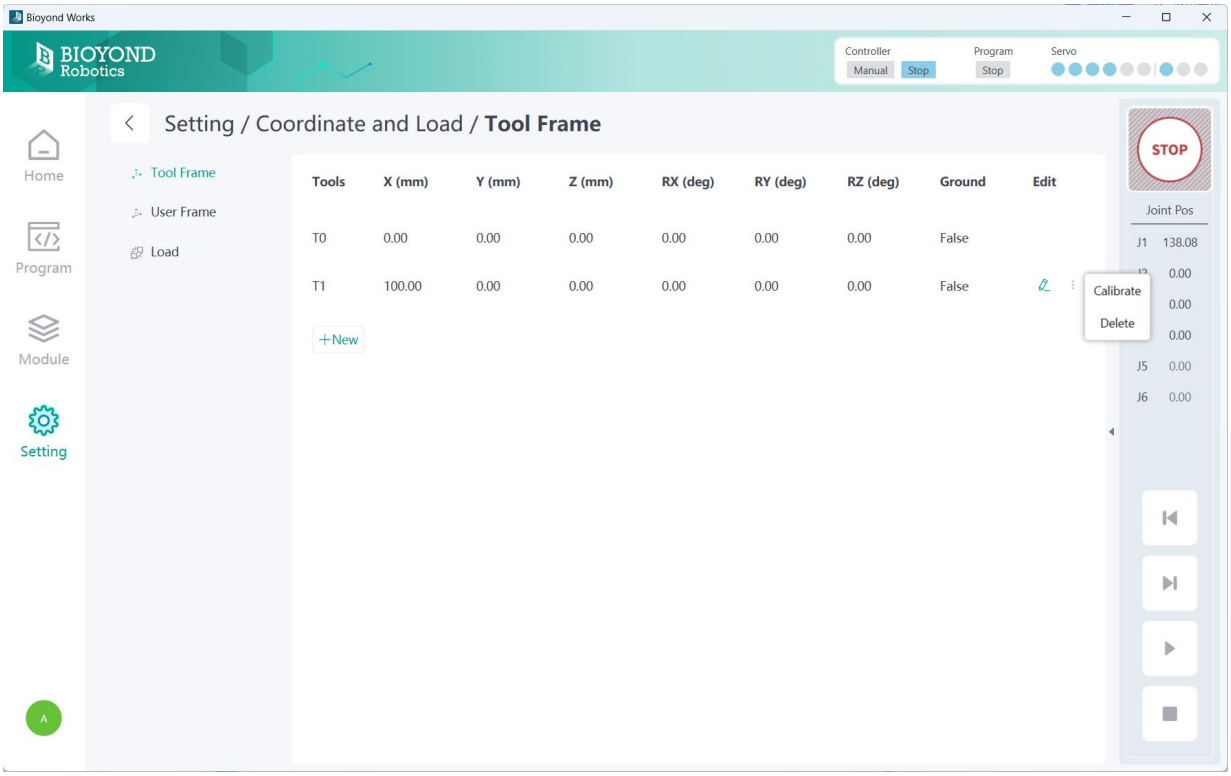
Follow the operation instructions, click "**Record**" button to record six points respectively, and then click "**Calibrate**" button to view the calibration results. After confirmation, click "Save" to make the calibration data effective.



■ Other operations

The T0 frame is the default tool frame and cannot be edited. Other tool frames involve operations such as editing, recalibrating, and deleting.



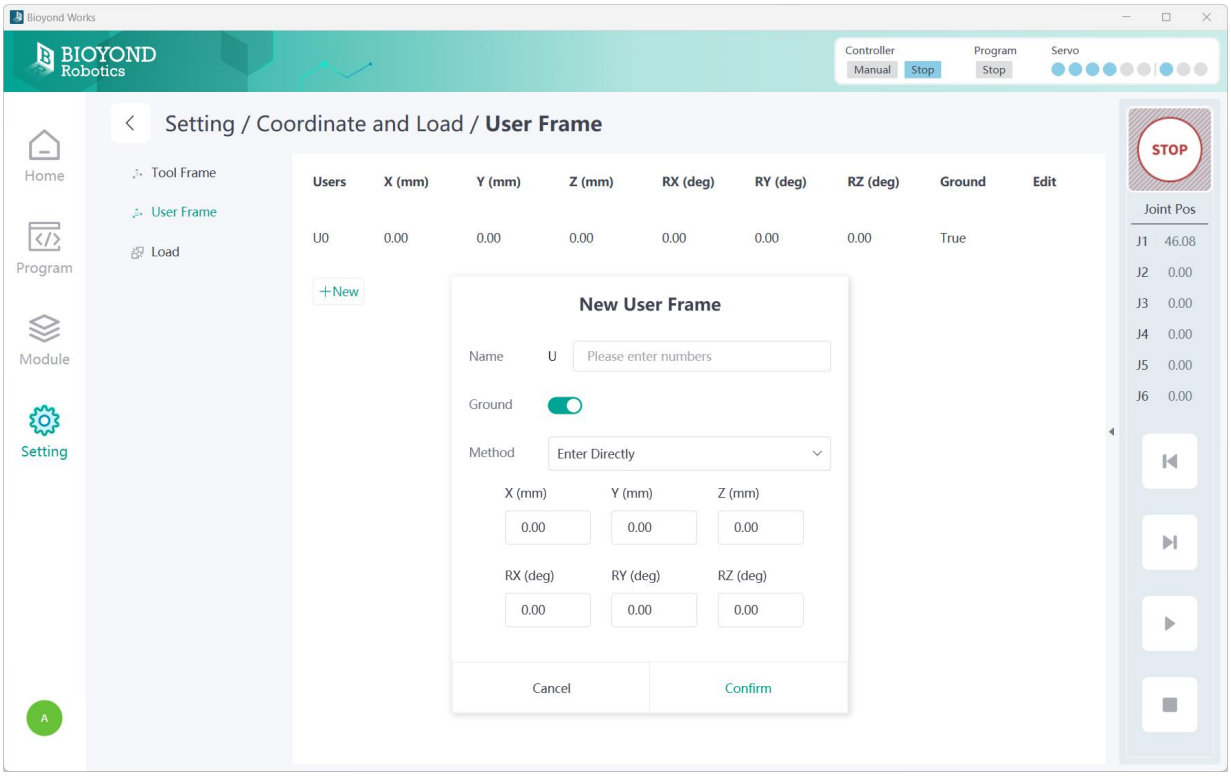


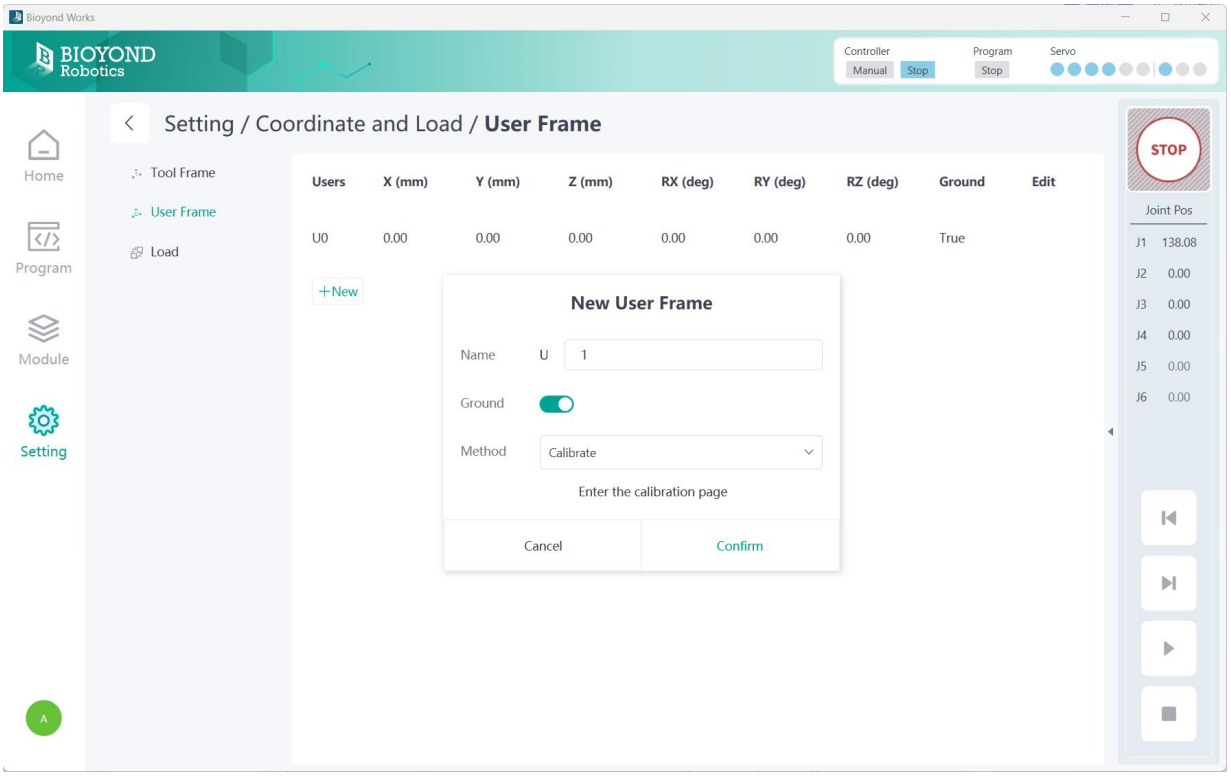
2) User frame

The user frame is established on the workpiece. When the position of the workpiece changes, only the user frame needs to be recalibrated, and the taught points within the program page will remain valid. The relevant operations are as follows:

■ New user frame

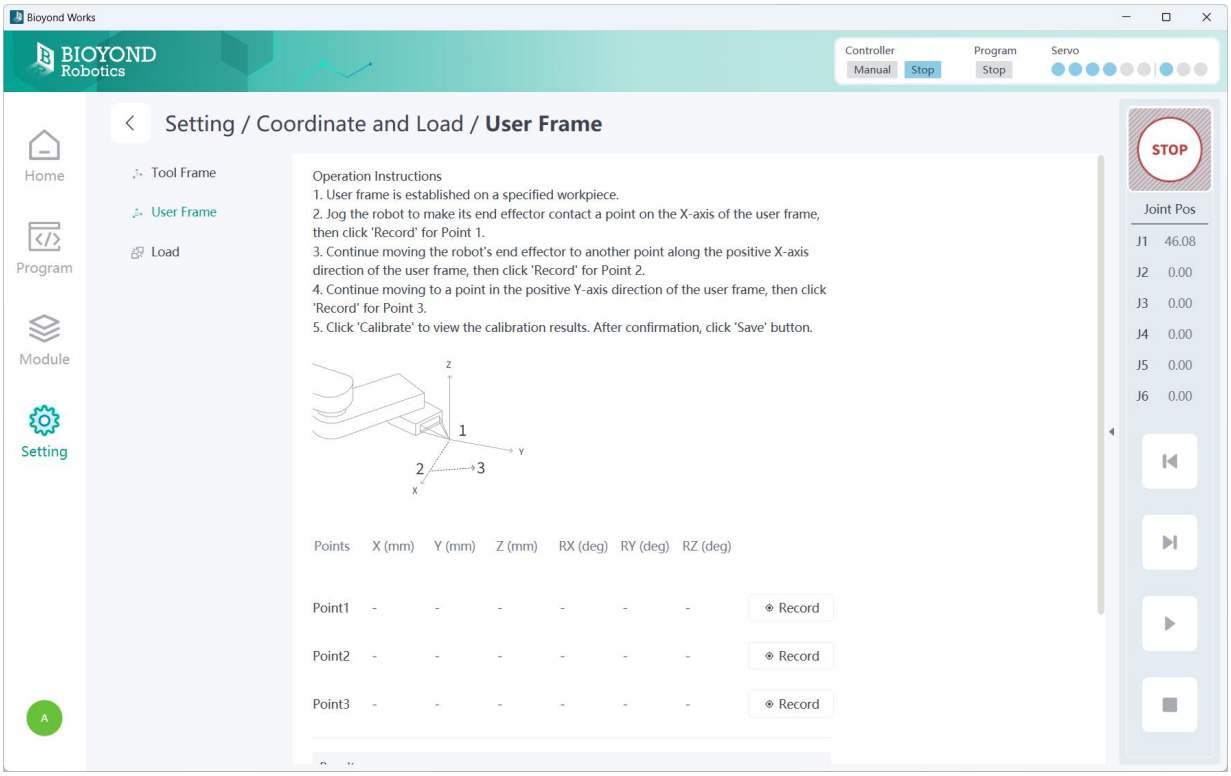
Click the " +New " button to bring up the new user frame window. Enter the name of the user frame, which only supports numbers characters. Currently, the user frame is defaulted to be fixed to the ground. Within the method, you can choose "**Direct Input**" or "**Calibrate**". If "**Calibrate**" is selected, you will proceed to the calibration page for further settings.





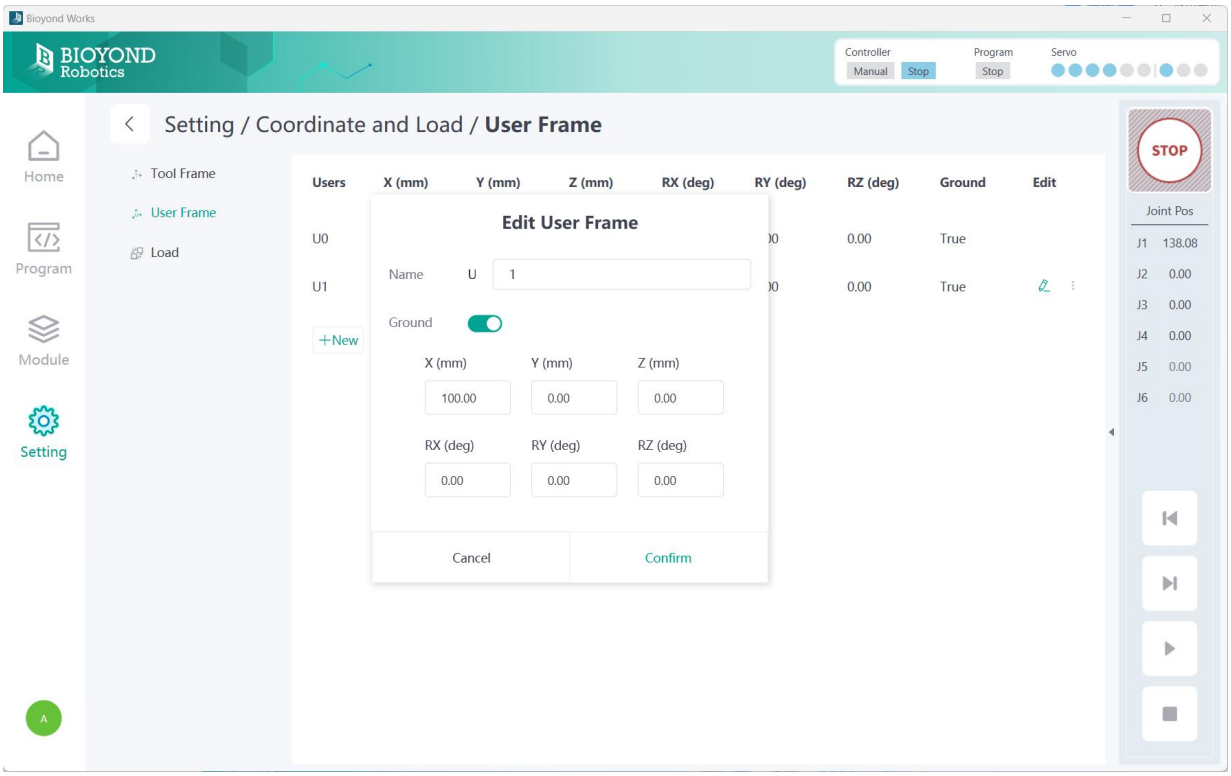
■ Calibrate

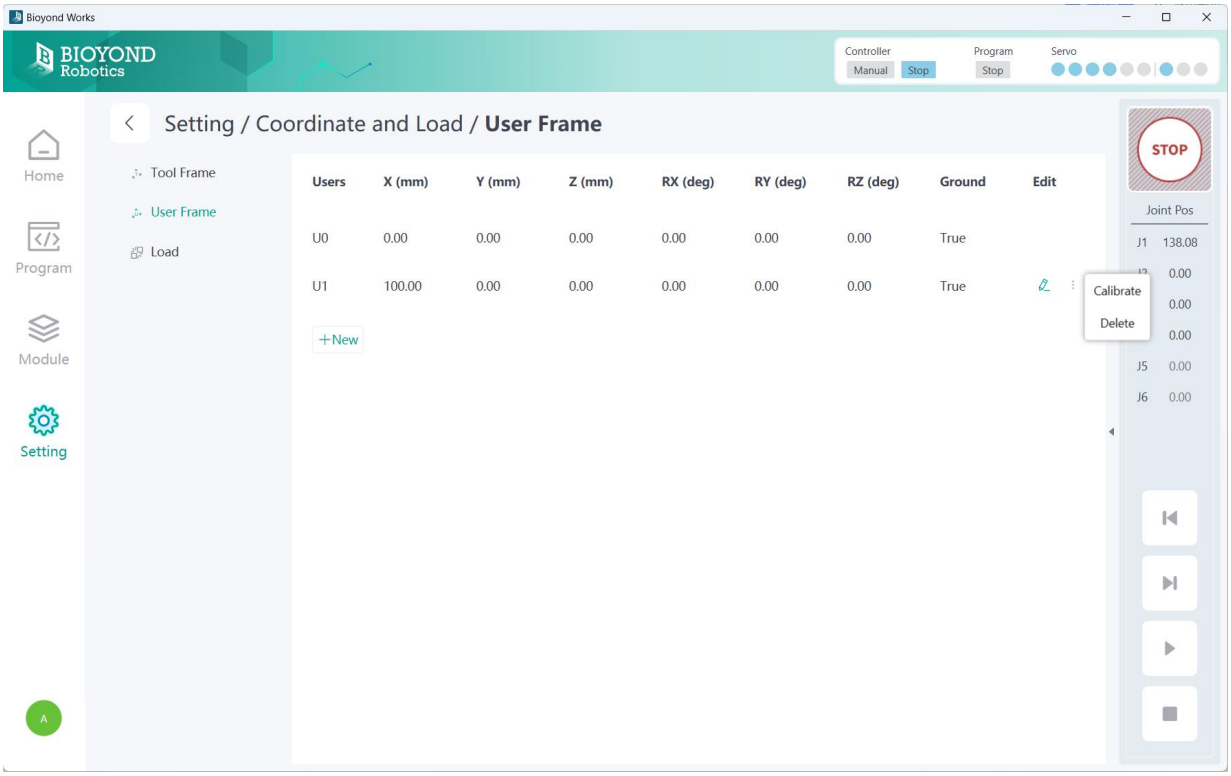
Follow the operation instructions, click "**Record**" button to record three points respectively, and then click "**Calibrate**" button to view the calibration results. After confirmation, click "Save" to make the calibration data effective.



■ Other operations

The U0 frame is the default user frame and cannot be edited. Other user frames involve operations such as editing, recalibrating, and deleting.





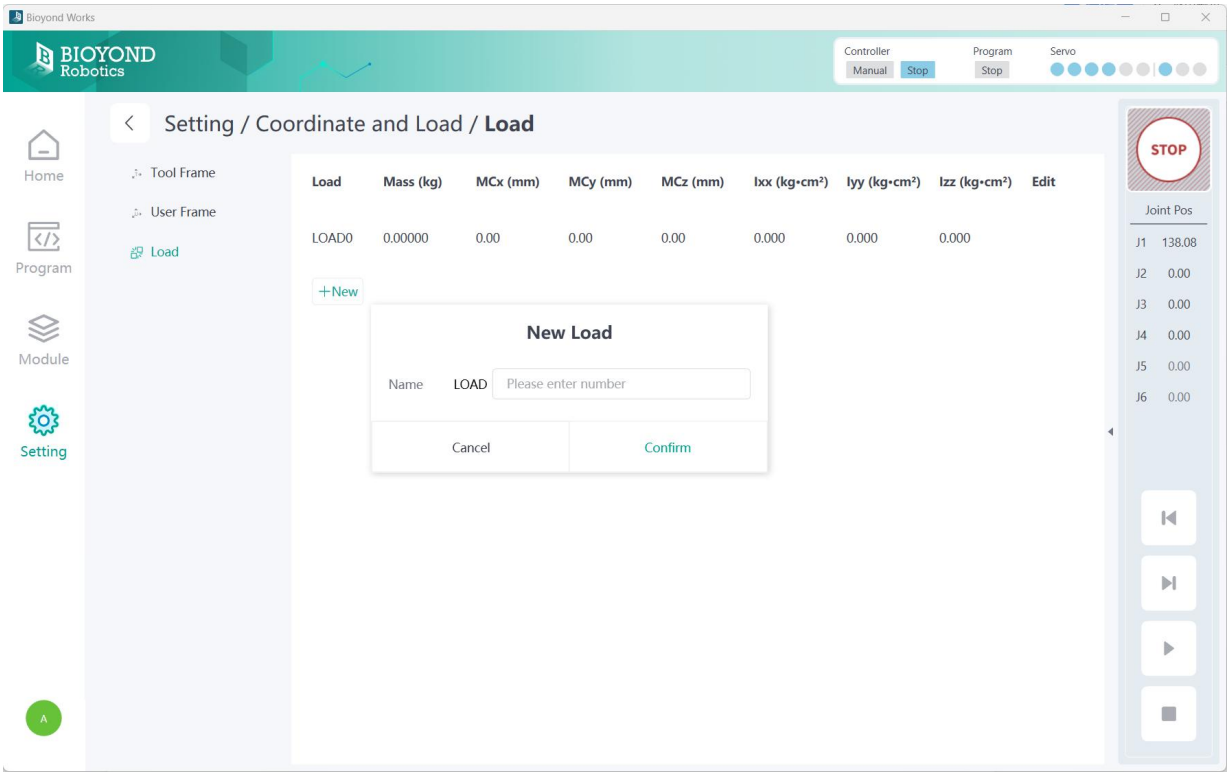
3) Load

Load refers to the information such as the mass, center of mass, and inertia tensor of all objects assembled at the end of a robot. Properly setting the load ensures that the robot performs excellently in various application scenarios.

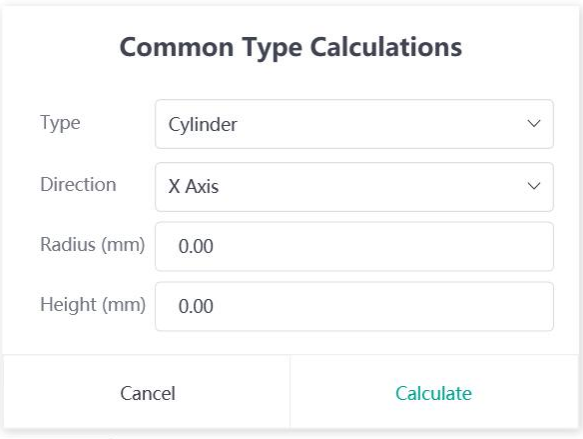
⚠ Warning: If the actual load differs from the set load, it may cause the robot to falsely report collisions or even cause damage to the robot.

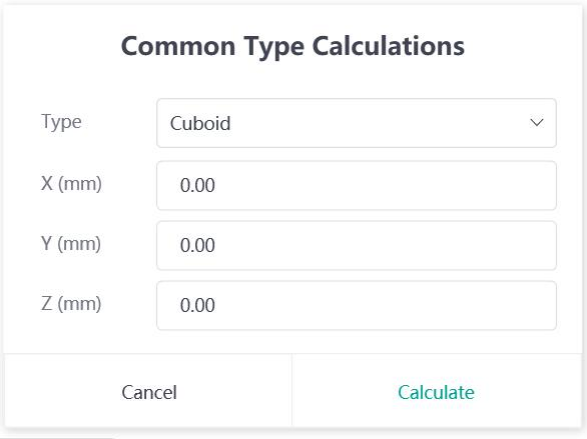
■ New load

Click the " +New " button to bring up the new load window. Enter the load name, which only supports numbers. After confirmation, you will proceed to the parameter settings page to continue the configuration.

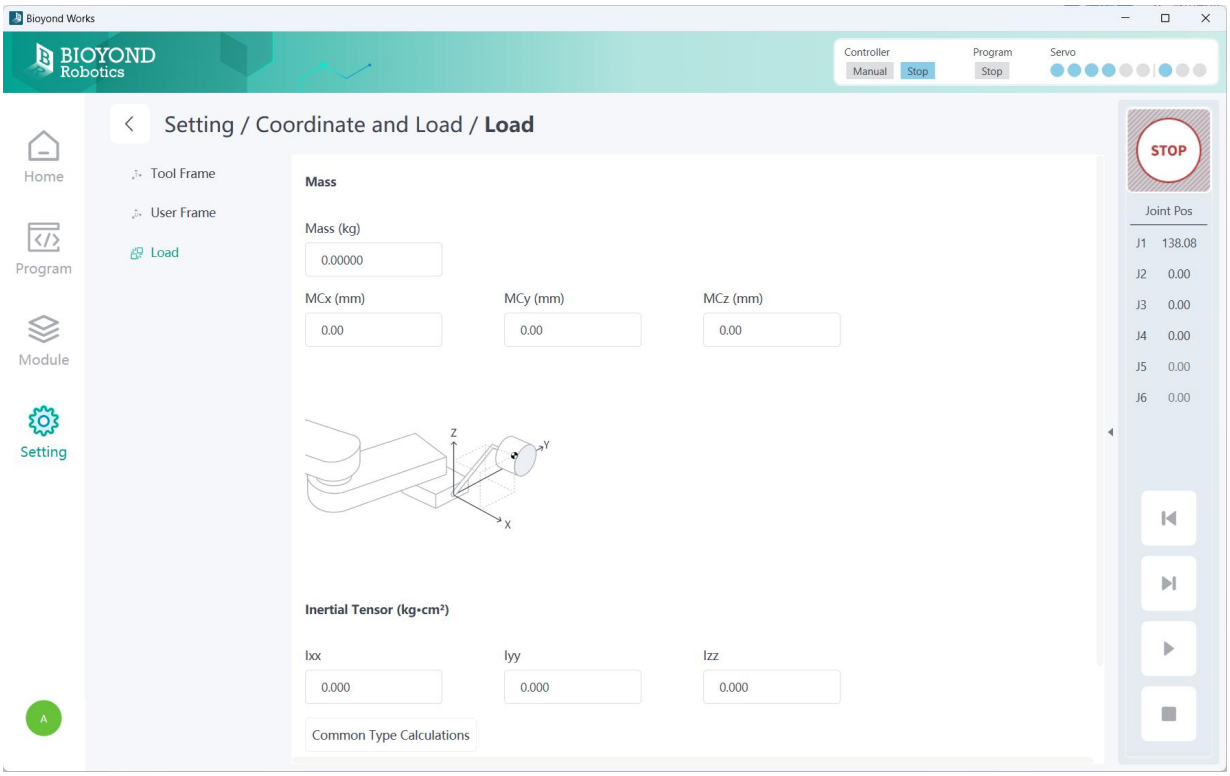


Enter the information for the load's mass, center of mass, inertia tensor of the rod, etc. Among these, you can use the "**Common Type Calculation**" to calculate the inertia tensor for cylinder or cuboid.



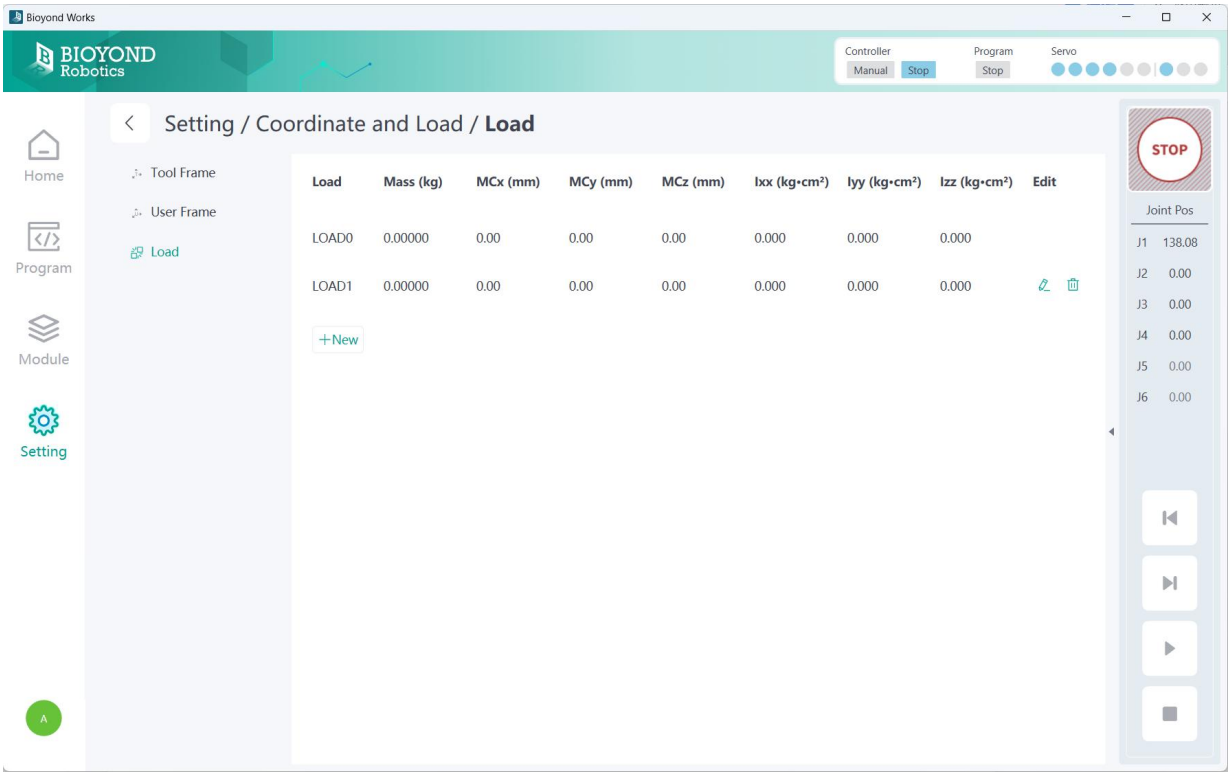


After confirmation, click "**Save**" to make the load data effective.



■ Other operations

LOAD0 is the default load and cannot be edited. Other loads can be edited and deleted.



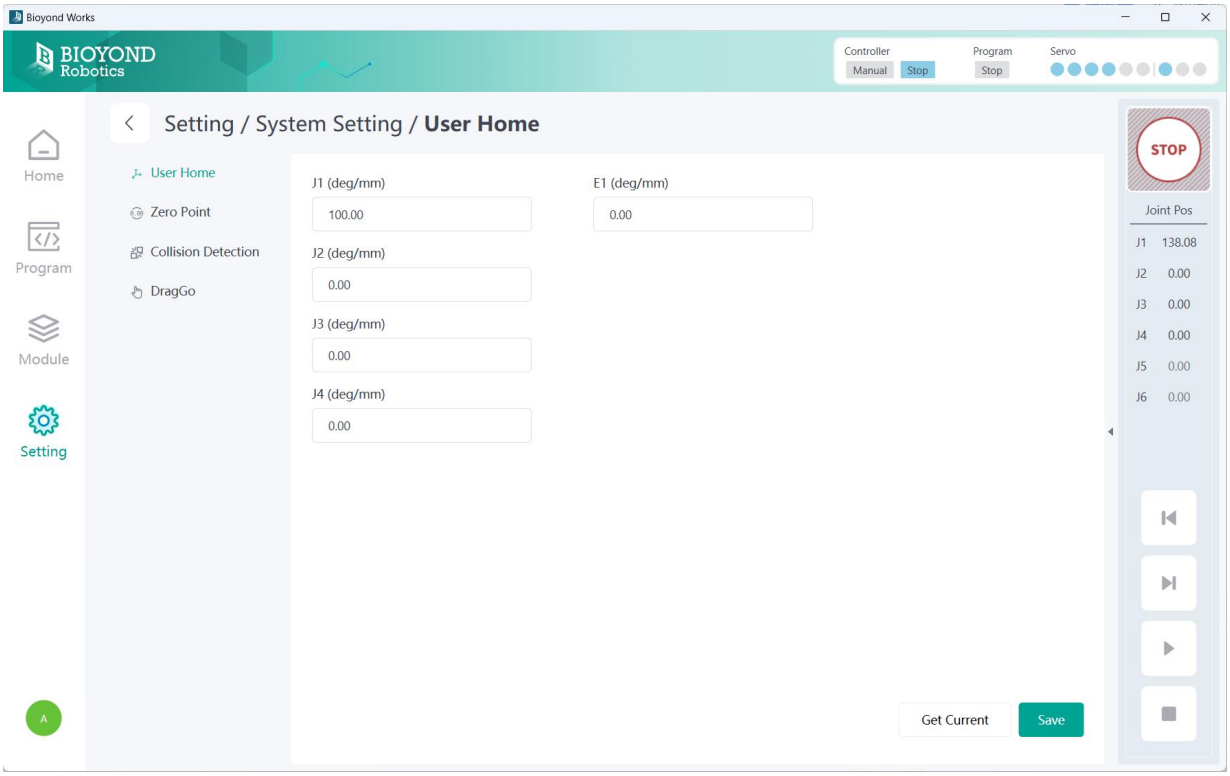
8.4. System setting

System Setting is used to configure parameters including user home, zero joint, collision detection and DragGo. The configurations will take effect only after rebooting the controller.

1) User home

The user home is a reference point selected by the user within the robot's movement range based on practical application scenarios and needs, this position can either be the robot's inital position or a user-defined position.

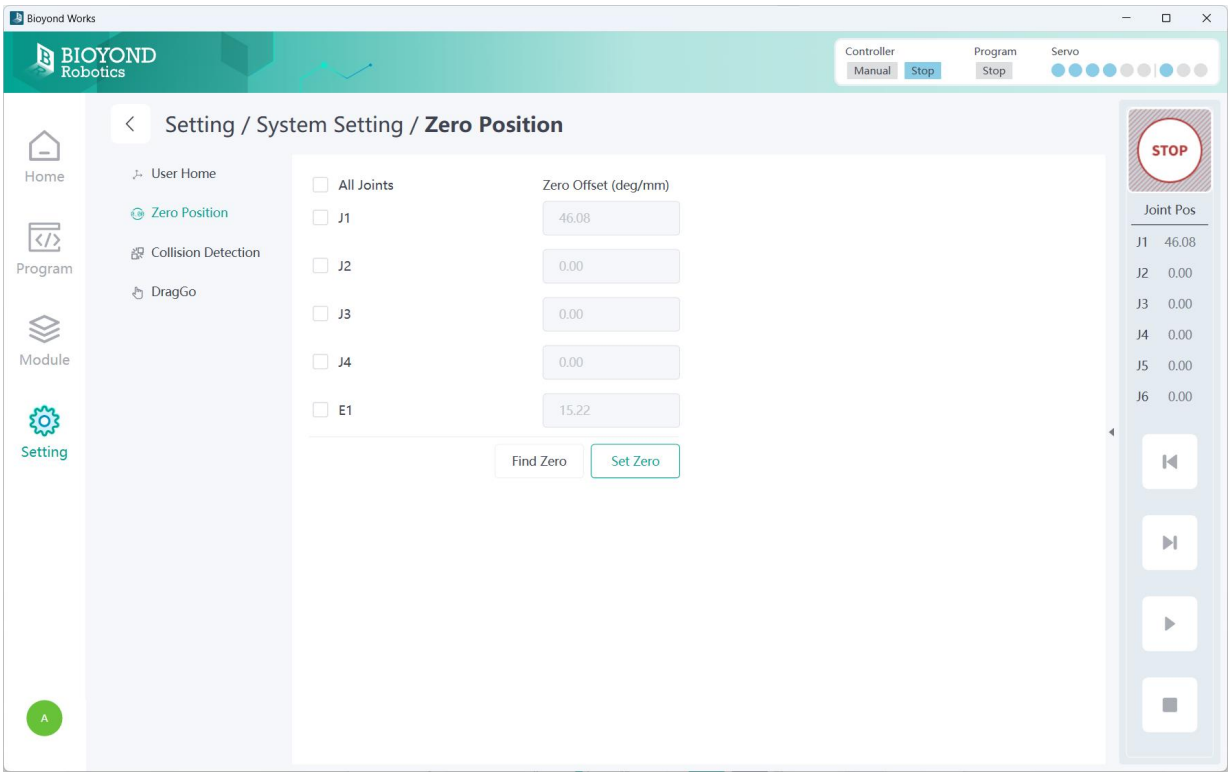
The user home can be set through the following page. First, adjust the joints to the desired return position use Jog control to adjust the points to the desired return position. Then, click "**Get Current**" to obtain the corresponding joint's position. Save the settings to make them effective.



2) Zero point

The zero joint position is a benchmark position determined for each joint of the robot at the factory or after specific adjustments.

In certain situations, such as when changing batteries, replacing mechanical system components, or after the robot has collided with a workpiece or the environment, it may be necessary to recalibrate the zero joint position. The calibration process typically involves moving each joint of the robot to a predetermined mechanical zero position and clicking on "Set Zero" to recalibrate the robot.

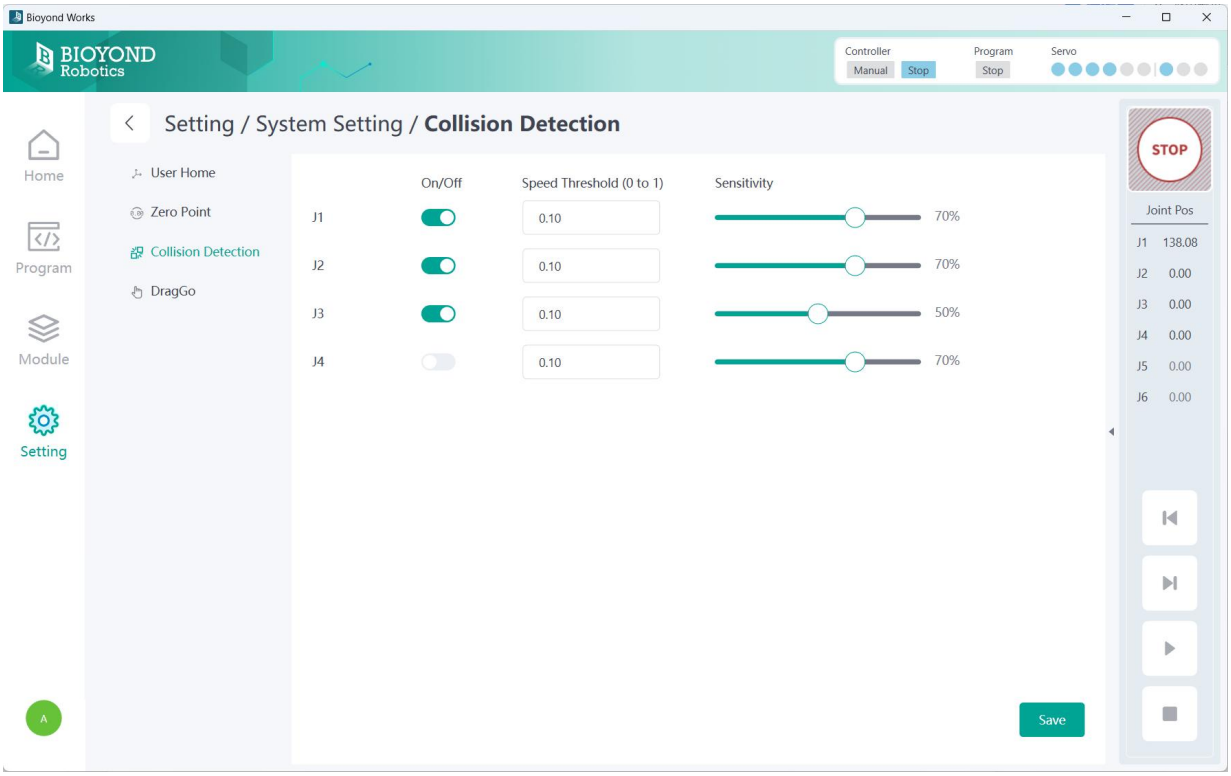


3) Collision detection

■ Enable Collision Detection: Choose enable or disable collision detection for the robot joints.

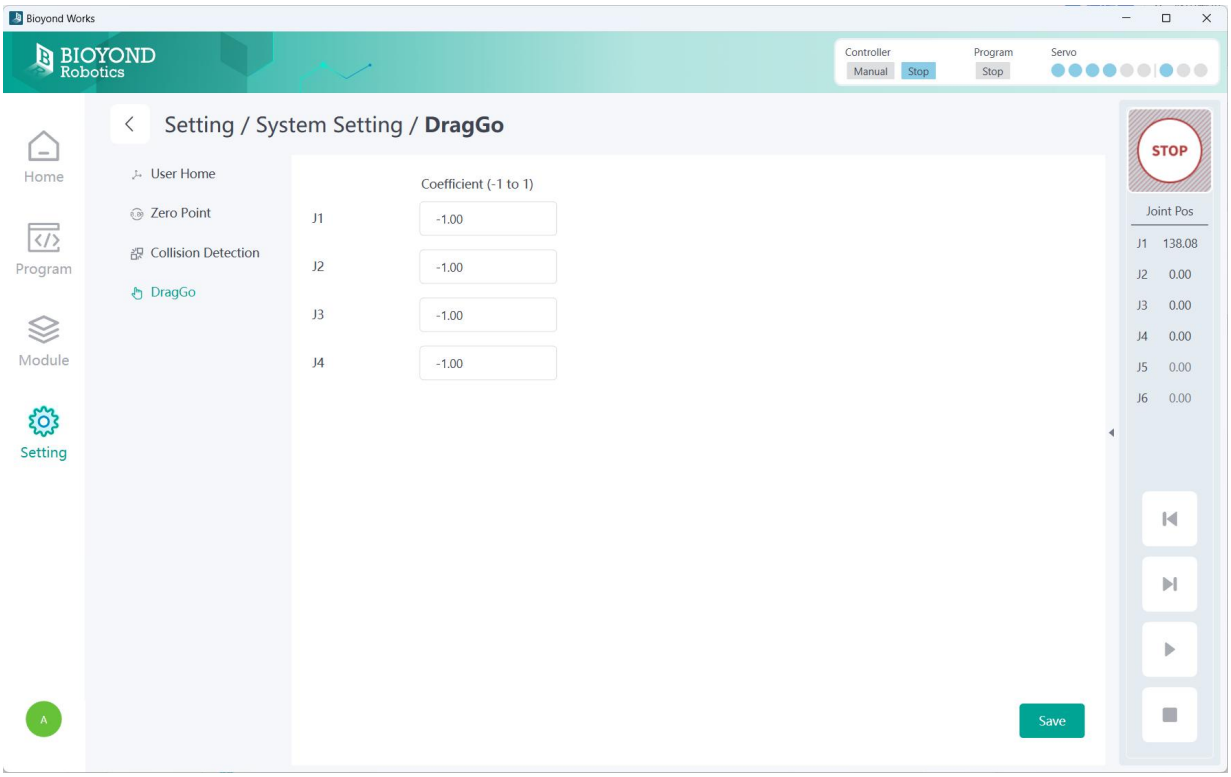
■ Trigger Speed Threshold: Set the speed threshold at which collision detection will be enabled automatically.

■ Sensitivity: Adjust the sensitivity level to determine the responsiveness to potential collisions.



4) DragGo

Configure the adjustment coefficient when manipulating robot joints. A higher coefficient makes dragging easier but increases inertia when stopping. The range is from -1 to 1.



8.5. Backup and Restore

1) Backup

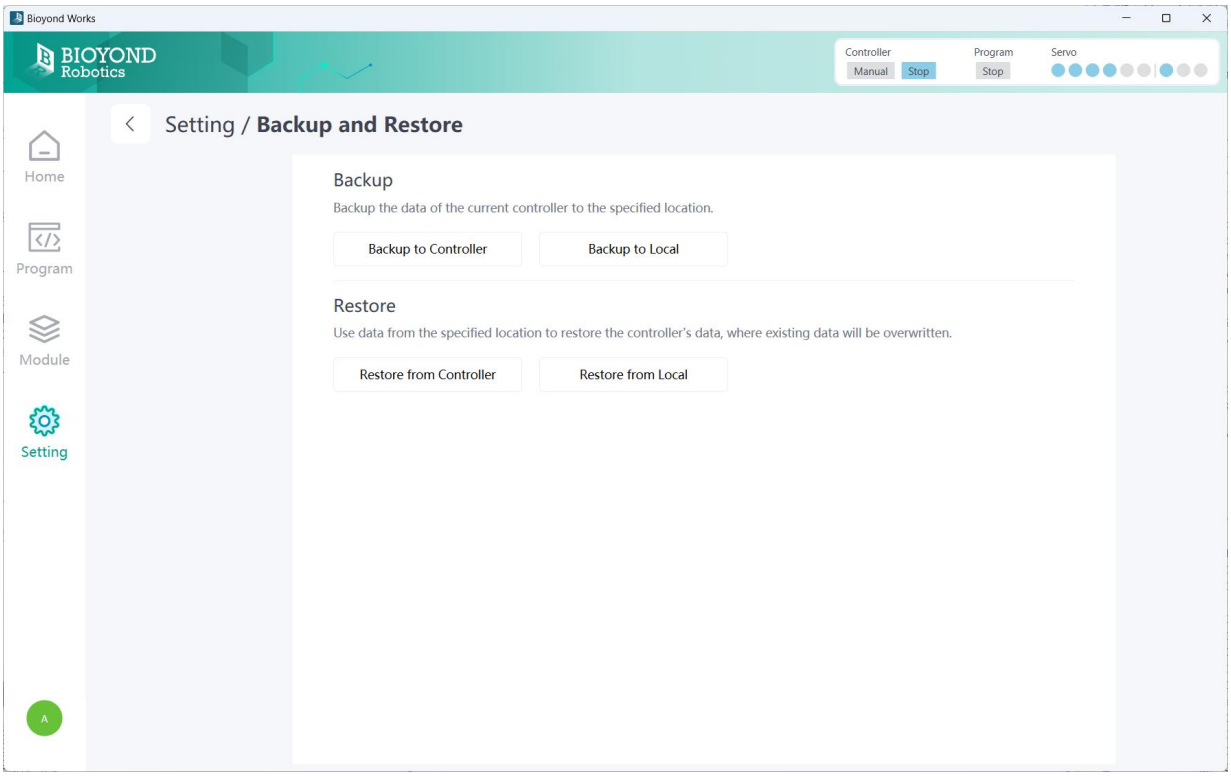
To ensure the stability of the robotic system and the integrity of data, it is essential to regularly backup configuration files, especially before making significant adjustments to LUA programs, before upgrading firmware, and before making major changes to system configurations. This helps to promptly restore the system to normal operating status in case of system failures or data loss.

Back up the data of the current controller to a specified location. "**Backup to Controller**" saves the data to the "backup" folder within the controller, with the file name recording the backup time, such as "backup_2024_12_10_11_15_02". "**Backup to Local**" saves the data to a specified path on the computer.

2) Restore

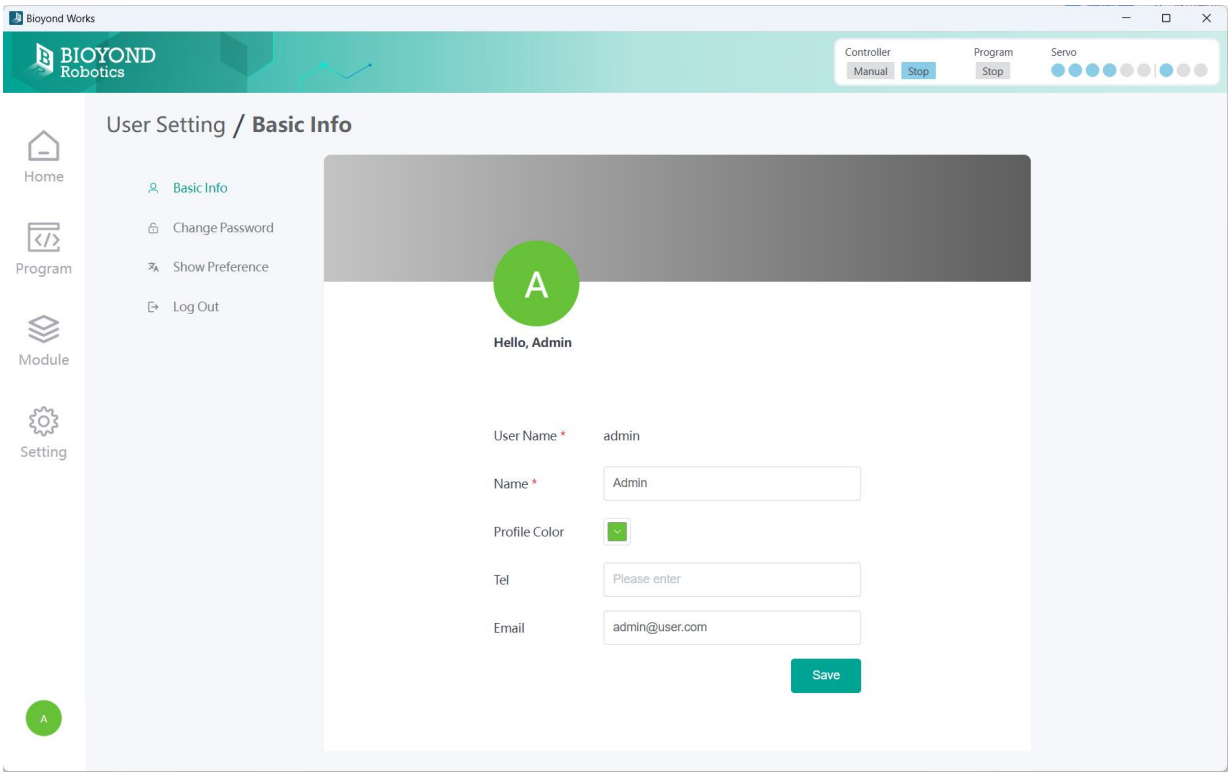
Restore the controller's data using the data from the specified location.

⚠ Warning: This operation will overwrite the current controller data. It is recommended to make a backup before restoring to avoid data loss.

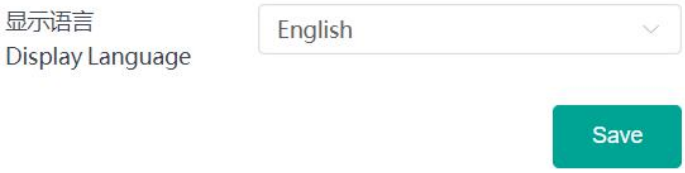

Bioyond Works
BIOYOND
Robotics
Controller
Manual
Stop
Program
Stop
Servo
Home
Program
Module
Setting
Setting / Backup and Restore
Backup
Backup the data of the current controller to the specified location.
Backup to Controller
Backup to Local
Restore
Use data from the specified location to restore the controller's data, where existing data will be overwritten.
Restore from Controller
Restore from Local

9. User Setting

In the Personal Center, you can edit the basic information of the current user, change the password, switch the display language, and log out.



"Show Preferences" can set the language of the software, currently supports Chinese and English.



Appendix A: Commands and Data Types

A.1 Generic Variable

In the program library, the variables defined in data.lua are global variables, which can be accessed by all subprograms.
Variables defined in the main program or subroutines are local variables, which are only valid within the current file and cannot be accessed by other subroutines.
Common variables include DefTool (tool frame), DefUser (user frame), DefLoad (load), DefArray (array), JointTarget (joint position data), and RobTarget (cartesian position data).

A.1.1 DefTool

Instruction format: DefTool(X, Y, Z, RX, RY, RZ, IsFixedGround)
Instruction function: Define tool frame.
Data meaning:
（1）X、Y、Z、RX、RY、RZ: Position and euler angles;
（2）IsFixedGround: Whether the ground is fixed.

A.1.2 DefUser

Instruction format: DefUser(X, Y, Z, RX, RY, RZ, IsFixedGround)
Instruction function: Define user frame.
Data meaning:
（1）X、Y、Z、RX、RY、RZ: Position and euler angles;
（2）IsFixedGround: Whether the ground is fixed.

A.1.3 DefLoad

Instruction format: DefLoad(MASS, MCX, MCY, MCZ, IXX, IYY, IZZ)
Instruction function: Define load data.
Data meaning:
（1）MASS: Mass of the load;
（2）MCX: Centroid in the X-direction;
（3）MCY: Centroid in the Y-direction;
（4）MCZ: Centroid in the Z-direction;
（5）IXX: XX-direction component of the lever inertia tensor;
（6）IYY: YY-direction component of the lever inertia tensor;
（7）IZZ: ZZ-direction component of the lever inertia tensor.

A.1.4 DefArray

Instruction format: DefArray(D1, D2, D3, D4, D5, D6)
Instruction function: Define 6 length array variables.
Data meaning:
（1）D1-D6: 6 array members.

A.1.5 JointTarget

Instruction format: JointTarget(J1, J2, J3, J4, J5, J6, E1, E2, E3, E4, E5, E6)

Instruction function: Joint variables.
Data meaning:
（1）J1-J6: 6 joint values;
（2）E1-E6: 6 external joint values.

A.1.6 JointTarget2

Instruction format: JointTarget2(LA, E1, E2, E3, E4, E5, E6)
Instruction function: Joint variables 2.
Data meaning:
（1）LA: 6 length array variable;
（2）E1-E6: 6 external joint values.

A.1.7 RobTarget

Instruction format: RobTarget(X, Y, Z, RX, RY, RZ, E1, E2, E3, E4, E5, E6)
Instruction function: Cartesian data.
Data meaning:
（1）X、Y、Z、RX、RY、RZ: Position and euler angles;
（2）E1-E6: 6 external joint values.

A.1.8 RobTarget2

Instruction format: RobTarget2(LA, E1, E2, E3, E4, E5, E6)
Instruction function: Cartesian data.
Data meaning:
（1）LA: 6 length position array;
（2）E1-E6: 6 external joint values.

A.2 Move Command

A.2.1 MoveJ

Instruction format: MoveJ(LP, VJ, ZJ, T0, U0)
Instruction function: Move the robot by joint.
Data meaning:
（1）LP: Cartesian position data;
（2）VJ: Speed data;
（3）ZJ: Zone data;
（4）T0: Tool frame data;
（5）U0: User frame data.

A.2.2 MoveL

Instruction format: MoveL(LP, V, ZP, T0, U0)
Instruction function: Move the robot linearly.
Data meaning:
（1）LP: Cartesian position data;
（2）V: Speed data;
（3）ZP: Zone data;
（4）T0: Tool frame data;
（5）U0: User frame data.

A.2.3 MoveC

Instruction format: MoveC(VP, LP, V, ZP, T0, U0)
Instruction function: Move the robot circularly.
Data meaning:
（1）VP: Circle point;
（2）LP: Target point;
（3）V: Speed data;
（4）ZP: Zone data;
（5）T0: Tool frame data;
（6）U0: User frame data.

A.2.4 MoveAbsJ

Instruction format: MoveAbsJ(LJ, VJ, ZJ)
Instruction function: Move the robot to an absolute joint position.
Data meaning:
（1）LJ: Joint position data;
（2）VJ: Speed data;
（3）ZJ: Zone data.

A.2.5 MoveExtJ

Instruction format: MoveExtJ(LJ,VJ,ZJ,T0,U0)
Instruction function: Move the robot by external joint.
Data meaning:
（1）LJ: Joint position data;
（2）VJ: Speed data;
（3）ZJ: Zone data;
（4）T0: Tool frame data;
（5）U0: User frame data.

A.2.6 MoveJump

Instruction format: MoveJump(LP, H, HUP, HDOWN, V, ZP, T0, U0)
Instruction function: Move the robot with a portal path.
Data meaning:
（1）LP: Cartesian position data;
（2）H: Maximum height;
（3）HUP: Ascending height;
（4）HDOWN: Descending height;
（5）V: Speed data;
（6）ZP: Zone data;
（7）T0: Tool frame data;
（8）U0: User frame data.

A.2.7 SetOffset

Instruction format: SetOffset(LP, X, Y, Z)
Instruction function: Set cartesian position offset.
Data meaning:
（1）LP: Cartesian position data;
（2）X: X offset;
（3）Y: Y offset;
（4）Z: Z offset.

A.2.8 MoveOffs

Instruction format: MoveOffs(LP, X, Y, Z, V, ZP, T0, U0)
Instruction function: Move the robot linearly with offset.
Data meaning:

（1）LP: Cartesian position data;
（2）X: X offse;
（3）Y: Y offset;
（4）Z: Z offset;
（5）V: Speed data;
（6）ZP: Zone data;
（7）T0: Tool frame data;
（8）U0: User frame data.

A.2.9 SetOffsetWithRz

Instruction format: SetOffsetWithRz(LP, X, Y, Z, RZ)
Instruction function: Set offset including RZ.
Data meaning:

（1）LP: Cartesian position data;
（2）X: X offse;
（3）Y: Y offse;
（4）Z: Z offse;
（5）RZ: RZ offse.

A.2.10 MoveLOffsWithRz

Instruction format: MoveLOffsWithRz(LP, X, Y, Z, RZ, V, ZP, T0, U0)
Instruction function: Move the robot linearly with offset including RZ.
Data meaning:

（1）LP: Cartesian position data;
（2）X: X offse;
（3）Y: Y offse;
（4）Z: Z offse;
（5）RZ: RZ offse;
（6）V: Speed data;
（7）ZP: Zone data;
（8）T0: Tool frame data;
（9）U0: User frame data.

A.2.11 MOVFINISH

Instruction format: MOVFINISH()
Instruction function: Wait for robot motion complete.

A.2.13 ShiftTool

Instruction format: ShiftTool(LA)
Instruction function: Set tool frame offset.
Data meaning:

（1）LA: Offsets in X, Y, Z, RX, RY, RZ of the tool frame.

A.2.14 ShiftUser

Instruction format: ShiftUser(LA)

Instruction function: Set user frame offset.
Data meaning:
（1）LA: Offsets in X, Y, Z, RX, RY, RZ of the user frame.

A.2.15 SetLoad

Instruction format: SetLoad(Load)
Instruction function: Set flange load.
Data meaning:
（1）Load: Load data.

A.2.16 SetSpeedScale

Instruction format: SetSpeedScale(V)
Instruction function: Set speed scale for robot motion.
Data meaning:
（1）V: Speed scale.

A.2.17 SetAcc

Instruction format: SetAcc(Acc, Jerk)
Instruction function: Set acc and jerk.
Data meaning:
（1）Acc: Acc data;
（2）Jerk: Jerk data.

A.2.18 SetAccTime

Instruction format: SetAccTime(T)
Instruction function: Set acceleration and deceleration time.
Data meaning:
（1）T: Time for acceleration and deceleration.

A.3 Logical Command

A.3.1 if

Instruction format: if( ) then
Instruction function: Determine whether the contents of the condition are satisfied.

A.3.2 elseif

Instruction format: elseif( ) then
Instruction function: Determine whether the contents of the condition are satisfied.

A.3.3 else

Instruction format: else
Instruction function: If the condition in the if statement is false, the program jumps to else and executes the instruction after else.

A.3.4 for

Instruction format: for LVAR= 1,2,1 do

Instruction function: A for loop that repeats the specified statement a number of times that can be controlled in the for statement.

Data meaning:

（1）LVAR: Command return value LVAR indicates the number of times that the program runs the loop when the judgment condition is met;

（2）“1”: The first parameter of the instruction indicates that the program runs from 1;

（3）“2”: The second parameter of the instruction indicates that the program is run twice;

（4）“1”: The third parameter of the instruction indicates that the program increments by 1 each time it runs.

A.3.5 while

Instruction format: while( ) do

Instruction function: A while loop checks whether a condition is true before executing a statement. When the condition is true, the program executes certain statements repeatedly.

A.3.6 return

Instruction format: return

Instruction function: End the current function.

A.3.7 break

Instruction format: break

Instruction function: Break out of a for or while loop.

A.3.8 end

Instruction format: end

Instruction function: Ending or exiting a command run.

A.3.9 Annotate

Instruction format: --

Instruction function: This command is used for comments.

A.4 System Command

A.4.1 Calling subroutine

Instruction format: mod

Instruction function: Call subroutine file for program jump,.

A.4.2 ShowInfo

Instruction format: ShowInfo(S)

Instruction function: Print information in the system log.

Data meaning:

（1）S: String type data or predefined variables.

A.4.3 GotoError

Instruction format: GotoError(S)
Instruction function: Print error message.
Data meaning:
（1）S: String type data.

A.4.4 SetLuaAppSta

Instruction format: SetLuaAppSta(Status)
Instruction function: Set the running status of LuaApp.
Data meaning:
（1）Status: Running status, 0 for uninitialized, 1 for initialized.

A.4.5 DelayMS

Instruction format: DelayMS(T)
Instruction function: Set the system to sleep for a specified time before running.
Data meaning:
（1）T: Sleep time, ranging from 0 to 999999, in ms.

A.4.6 PAUSE

Instruction format: PAUSE()
Instruction function: Pause program execution at the current location.

A.4.7 GetTime

Instruction format: GetTime()
Instruction function: Get the system time.

A.4.8 SaveData

Instruction format: SaveData(Data)
Instruction function: Save data.
Data meaning:
（1）Data: User-defined integer data.

A.4.9 LoadData

Instruction format: LoadData()
Instruction function: Read data saved by SaveData.

A.4.10 GetElem

Instruction format: GetElem(LVAR, N)
Instruction function: Read the value of a sub-element.
Data meaning:
（1）LVAR: The operand, editable as an DefArray, JointTarget, or RobTarget value defined in data.lua or the current file;
（2）N: The element position, integer from 1 to 12.

A.4.11 SetElem

Instruction format: SetElem(LVAR, N, Data)
Instruction function: Set the value of a sub-element.
Data meaning:
（1）LVAR: The operand, editable as an DefArray, JointTarget, or RobTarget value defined in data.lua or the current file;
（2）N: The element position, integer from 1 to 12;
（3）Data: The modified value of the element.

A.4.12 GetPose

Instruction format: GetPose(LP, N)
Instruction function: Read the value of cartesian pose element.
Data meaning:
（1）LP: Cartesian position data;
（2）N: Fillable as X, Y, Z, RX, RY, RZ.

A.4.13 SetPose

Instruction format: SetPose(LP, N, Data)
Instruction function: Set the value of a cartesian pose element.
Data meaning:
（1）LP: Cartesian position data;
（2）N: Fillable as X, Y, Z, RX, RY, RZ;
（3）Data: The modified value of the element.

A.4.14 GetCRobT

Instruction format: GetCRobT(T0, U0)
Instruction function: Get current cartesian position.
Data meaning:
（1）T0: Tool frame data;
（2）U0: User frame data.

A.4.15 GetCJoints

Instruction format: GetCJoints()
Instruction function: Get current joint position.

A.4.16 GetCRefJoints

Instruction format: GetCRefJoints()
Instruction function: Get current target joint position.

A.4.17 GetCJointsVel

Instruction format: GetCJointsVel()
Instruction function: Get current joint velocity.

A.4.18 GetCJointsTrq

Instruction format: GetCJointsTrq()
Instruction function: Get current joint torque.

A.4.19 GetCJointsNormTrq

Instruction format: GetCJointsNormTrq()
Instruction function: Get normalized joint torque.

A.4.20 Check4JointsEqual

Instruction format: Check4JointsEqual(LJ1, LJ2, Error)
Instruction function: Whether the J1-J4 errors of the two joint variables are within a certain range.
Data meaning:
（1）LJ1: Joint variable 1;
（2）LJ2: Joint variable 2;
（3）Error: The acceptable error range.

A.4.21 Check6JointsEqual

Instruction format: Check6JointsEqual(LJ1, LJ2, Error)
Instruction function: Whether the J1-J6 errors of the two joint variables are within a certain range.
Data meaning:
（1）LJ1: Joint variable 1;
（2）LJ2: Joint variable 2;
（3）Error: The acceptable error range.

A.4.22 CheckRobTEqual

Instruction format: CheckRobTEqual(LP1, LP2, TError, RError)
Instruction function: Whether two cartesian variables are close within a certain range.
Data meaning:
（1）LP1: Cartesian variable 1;
（2）LP2: Cartesian variable 2;
（3）TError: Translational tolerance;
（4）RError: Rotational tolerance.

A.4.23 CollSupvOn

Instruction format: CollSupvOn()
Instruction function: Enable collision detection.

A.4.24 CollSupvOff

Instruction format: CollSupvOff()
Instruction function: Disable collision detection.

A.4.25 EnableRecordIns

Instruction format: EnableRecordIns()
Instruction function: Enable path recording.

A.4.26 DisableRecordIns

Instruction format: DisableRecordIns()
Instruction function: Disable path recording.

A.4.27 ReadReg

Instruction format: ReadReg(ID)
Instruction function: Read the value of the specified ID register.
Data meaning:
（1）ID: Register ID, an integer ranging from 2048 to 2119.

A.4.28 WriteReg

Instruction format: WriteReg(ID, LVAR)
Instruction function: Modify the value of the specified ID register.
Data meaning:
（1）ID: Register ID, an integer ranging from 2048 to 2119;
（2）LVAR: Editable as an integer ranging from 0 to 65535, a VAR value defined in data.lua, or a VAR value defined in the current file.

A.4.29 WaitReg

Instruction format: WaitReg(ID, OFF/ON, T)
Instruction function: Wait for the register to reach a certain value.
Data meaning:
（1）ID: Register ID, an integer ranging from 2048 to 2119;
（2）OFF/ON: The expected value of the virtual DI, 0 for OFF and 1 for ON;
（3）T: Timeout, an integer ranging from 0 to 999999, ms.

A.5 IO Command

A.5.1 ReadDI

Instruction format: ReadDI(IN)
Instruction function: Read the value of the DI.
Data meaning:
（1）IN: DI pin.

A.5.2 WriteDO

Instruction format: WriteDO(OT, OFF/ON)
Instruction function: Write the signal to the DO pin.
Data meaning:
（1）OT: DO pin;
（2）OFF/ON: State of the DO signal, 0 for OFF and 1 for ON.

A.5.3 WaitDI

Instruction format: WaitDI(IN, OFF, T)
Instruction function: Wait the DI to reach a set value.
Data meaning:
（1）IN: DI pin;
（2）OFF/ON: Set value of the DI, 0 for OFF and 1 for ON;
（3）T: timeout(ms), 0-999999.

A.5.4 WritePulseDO

Instruction format: WritePulseDO(OT, T)
Instruction function: Set the DO to ON for a while and set it to OFF after.
Data meaning:
（1）OT: DO pin;
（2）T: Time(ms) duration for the DO to remain ON, ranging from 0 to 999999.

A.5.5 ReadAI

Instruction format: ReadAI(IN)
Instruction function: Read the AI value.
Data meaning:
（1）IN: AI pin.

A.5.6 WriteAO

Instruction format: WriteAO(OT, Data)
Instruction function: Write AO.
Data meaning:
（1）OT: AO pin;
（2）Data: AO value.

A.5.7 ReadVirDI

Instruction format: ReadVirDI(IN)
Instruction function: Read the virtual DI.
Data meaning:
（1）IN: Virtual DI pin.

A.5.8 WriteVirDO

Instruction format: WriteVirDO(OT, OFF/ON)
Instruction function: Write the virtual DO.
Data meaning:
（1）OT: Virtual DO pin;
（2）OFF/ON: State of the specified DO pin, 0 for OFF and 1 for ON.

A.5.9 WaitVirDI

Instruction format: WaitVirDI(IN, OFF, T)
Instruction function: Wait the virtual DI to reach a set value.
Data meaning:
（1）IN: Virtual DI pin;
（2）OFF/ON: Set value of the virtual DI, 0 for OFF and 1 for ON;
（3）T: timeout , an integer ranging from 0 to 999999, ms.

A.5.10 WritePulseVirDO

Instruction format: WritePulseVirDO(OT, T)
Instruction function: Set the virtual DO to ON for a while, and set to OFF after.
Data meaning:
（1）OT: Virtual DO pin;
（2）T: Time duration for the DO to remain ON, ranging from 0 to 999999, ms.

A.6 Operation Command

The operation instruction includes four operations of variables and trigonometric function operations.

A.6.1 INC

Instruction format: INC(LVAR)
Instruction function: Increment operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.2 DEC

Instruction format: INC(LVAR)
Instruction function: Decrement operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.3 SIN

Instruction format: SIN(LVAR)
Instruction function: Sine operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.4 COS

Instruction format: COS(LVAR)
Instruction function: Cosine operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.5 TAN

Instruction format: TAN(LVAR)
Instruction function: Tangent operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.6 ASIN

Instruction format: ASIN(LVAR)
Instruction function: Arcsine operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.7 ACOS

Instruction format: ACOS(LVAR)
Instruction function: Arccosine operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.8 ATAN

Instruction format: ATAN(LVAR)
Instruction function: Arctangent operation.
Data meaning:
（1）LVAR: The data to be processed.

A.6.9 SQRT

Instruction format: SQRT(LVAR)
Instruction function: Square root operation.
Data meaning:
（1）LVAR: The data to be processed.

A.7 Bit Operation Command

A.7.1 BitAnd

Instruction format: BitAnd(N1,N2)
Instruction function: Logical AND operation.

A.7.2 BitOr

Instruction format: BitOr(N1, N2)
Instruction function: Logical OR operation.

A.7.3 BitNeg

Instruction format: BitNeg(N1)
Instruction function: Logical NOT operation.

A.7.4 BitXOr

Instruction format: BitXOr(N1, N2)
Instruction function: Logical XOR operation.

A.7.5 BitClear

Instruction format: BitClear(N1, N2)
Instruction function: Set the bit of the operand to 0.
Data meaning:
（1）N1: Operand ranging from 0 to 999999 defined in data.lua or current file;
（2）N2: The bit position to be operated ranging from 1 to 32.

A.7.6 BitSet

Instruction format: BitSet(N1, N2)
Instruction function: Set the bit of the operand to 1.
Data meaning:
（1）N1: Operand ranging from 0 to 999999 defined in data.lua or current file;
（2）N2: The bit position to be operated ranging from 1 to 32.

A.7.7 BitCheck

Instruction format: BitCheck(N1, N2)
Instruction function: Check if the bit of the operand is 1.
Data meaning:
（1）N1: Operand ranging from 0 to 999999 defined in data.lua or current file;
（2）N2: The bit position to be operated ranging from 1 to 32.

A.7.8 BitLSh

Instruction format: BitLSh(N1, N2)
Instruction function: Bitwise left shift operation.
Data meaning:
（1）N1: Operand ranging from 0 to 999999 defined in data.lua or current file;
（2）N2: The number to be shifted, ranging from 1 to 32.

A.7.9 BitRSh

Instruction format: BitRSh(N1, N2)
Instruction function: Bitwise right shift operation.
Data meaning:
（1）N1: Operand ranging from 0 to 999999 defined in data.lua or current file;
（2）N2: The number to be shifted, ranging from 1 to 32.

A.8 System Variable Command

A.8.1 GetSysVarI

Instruction format: GetSysVarI()
Instruction function: Get the value of system input variable.

A.8.2 SetSysInVarI

Instruction format: SetSysInVarI(N, Data)
Instruction function: Set the value of int system input variable.
Data meaning:
（1）N: System variable;
（2）Data: Value.

A.8.3 SendSysOutVarI

Instruction format: SendSysOutVarI(N, Data)
Instruction function: Set the value of int system output variable.
Data meaning:
（1）N: System variable;
（2）Data: Value.

A.8.4 ResetSysOutVarI

Instruction format: ResetSysOutVarI()
Instruction function: Reset all int system output variables.

A.8.5 GetSysParamI

Instruction format: GetSysParamI(N)
Instruction function: Get int system parameter.
Data meaning:
（1）N: System parameter.

A.8.6 GetSysParamF

Instruction format: GetSysParamF(N)
Instruction function: Get float system parameter.
Data meaning:
（1）N: System parameter.

A.9 Stack Command

A.9.1 StackConfiguration

Instruction format: StackConfiguration(Row, Col, P1, P2, P3, P4)
Instruction function: Set stack table.
Data meaning:
（1）Row: Number of stack rows;
（2）Col: Number of stack columns;
（3）P1-P4: 4 reference points of the stack.

A.9.2 GetStackTargetPoint

Instruction format: GetStackTargetPoint(Stack, Row, Col)
Instruction function: Get slot position of stack.
Data meaning:
（1）Stack: Stack table, obtained from the instruction StackConfiguration;
（2）Row: The row of stack;
（3）Col: The column of stack.

A.9.3 Stack2Configuration

Instruction format: Stack2Configuration(ObjId, StackHeight, OffsetHeight)
Instruction function: Configure single-column stack table for material.
Data meaning:
（1）ObjId: Material ID;
（2）StackHeight: Stacking height of the material;
（3）OffsetHeight: Pick-up height of the material.

A.9.4 GetStack2TargetPoint

Instruction format: GetStack2TargetPoint(Stack, PointIndex, LowestPoint, OffsetX, OffsetY)
Instruction function: Get stack position based on index.
Data meaning:
（1）Stack: Stack table;
（2）Index: Target point index;
（3）FirstPoint: The first point of the target column;
（4）OffsetX: Offset in the X direction for the target point, default is 0;
（5）OffsetY: Offset in the Y direction for the target point, default is 0.

A.9.5 GetStack2Group

Instruction format: GetStack2Group()
Instruction function: Configure single-column or multi-column stack table group for material.

A.9.6 SearchInStack2Group

Instruction format: SearchInStack2Group(ObjId)
Instruction function: Get stack table ID based on material ID.
Data meaning:
（1）ObjId: Material ID.

A.10 Gripper Configuration Command

A.10.1 EnableGripperDet

Instruction format: EnableGripperDet(N)
Instruction function: Enable gripper hollow detection.
Data meaning:
（1）N: Gripper module Index.

A.10.2 DisableGripperDet

Instruction format: DisableGripperDet(N)
Instruction function: Disable gripper hollow detection.
Data meaning:
（1）N: Gripper module Index.

A.10.3 SetExtJTrqLimit

Instruction format: SetExtJTrqLimit(ForceArray)
Instruction function: Set the torque limit for the external axis joint.
Data meaning:
（1）ForceArray: Torque threshold.

A.10.4 MoveExtJWithForce

Instruction format: MoveExtJWithForce(NumArray, V, ForceArray)
Instruction function: Move the robot with external joint and set torque.
Data meaning:
（1）NumArray: External axis number array;
（2）V: Maximum motion velocity;
（3）ForceArray: Force array.

A.10.5 GripperEmptyDet

Instruction format: GripperEmptyDet(N, TrqThr, DisThr)
Instruction function: Gripper configuration.
Data meaning:
（1）N: Number of the external axis of the gripper;
（2）TrqThr: Torque threshold;
（3）DisThr: Distance difference threshold(actual value - target value), the target value is

generally set to 0.

A.10.6 SetGripperTorqueLimit

Instruction format: SetGripperTorqueLimit(N, DofIndex, Torque)
Instruction function: Set the force of the gripper.
Data meaning:
（1）N: Gripper module number;
（2）DofIndex: Degree of freedom index;
（3）Torque: Torque percentage value.

A.10.7 SetGripperTorqueThreshold

Instruction format: SetGripperTorqueThreshold(N, AxisIndex, TrqThr)
Instruction function: Set the torque threshold(in percentage) for the gripper.
Data meaning:
（1）N: Gripper module number;
（2）AxisIndex: External axis number of the gripper;
（3）TrqThr: Torque threshold in percentage.

A.10.8 SetGripperPositionThreshold

Instruction format: SetGripperPositionThreshold(N, AxisIndex, DisThr)
Instruction function: Set the threshold of position difference for the gripper.
Data meaning:
（1）N: Gripper module number;
（2）AxisIndex: External axis number of the gripper;
（3）DisThr: Threshold of position difference, in m.

A.11 ThirdParty Gripper Command

A.11.1 CreatThirdPartyGripperEntity

Instruction format: CreatThirdPartyGripperEntity(ChannelId, DevTypeId, DevId, Baudrate)
Instruction function: Gripper Initialization.
Data meaning:
（1）ChannelId: UART channel number;
（2）DevTypeId: Gripper type ID;
（3）DevId: Gripper ID;
（4）Baudrate: Baud rate.

A.11.2 ThirdPartyGripperEnable

Instruction format: ThirdPartyGripperEnable()
Instruction function: Gripper enable.

A.11.3 ThirdPartyGripperDisable

Instruction format: ThirdPartyGripperDisable()
Instruction function: Gripper disable.

A.11.4 ThirdPartyGripperGetFaultMsg

Instruction format: ThirdPartyGripperGetFaultMsg()
Instruction function: Get gripper error information.

A.11.5 ThirdPartyGripperOpenFast

Instruction format: ThirdPartyGripperOpenFast()
Instruction function: Open gripper with full speed and force, default to maximum opening.

A.11.6 ThirdPartyGripperOpenNormal

Instruction format: ThirdPartyGripperOpenNormal()
Instruction function: Open gripper with half speed and force, default to maximum opening.

A.11.7 ThirdPartyGripperOpenSlow

Instruction format: ThirdPartyGripperOpenSlow()
Instruction function: Open gripper at low speed and force, default to maximum opening.

A.11.8 ThirdPartyGripperCloseFast

Instruction format: ThirdPartyGripperCloseFast()
Instruction function: Close gripper at full speed and force, default to minimum clamping.

A.11.9 ThirdPartyGripperCloseNormal

Instruction format: ThirdPartyGripperCloseNormal()
Instruction function: Close gripper at half speed and force, default to minimum clamping.

A.11.10 ThirdPartyGripperCloseSlow

Instruction format: ThirdPartyGripperCloseSlow()
Instruction function: Close gripper at low speed and force, default to minimum clamping.

A.11.11 ThirdPartyGripperControl

Instruction format: ThirdPartyGripperControl(Position, Speed, Torque)
Instruction function: Gripper control.
Data meaning:

（1）Position: Position percentage;
（2）Speed: Speed percentage;
（3）Torque: Torque percentage.

A.11.12 GetThirdPartyGripperPos

Instruction format: GetThirdPartyGripperPos()
Instruction function: Get the current position of the gripper.

A.11.13 UnmountThirdPartyGripperEntity

Instruction format: UnmountThirdPartyGripperEntity()
Instruction function: Uninstall the third-party gripper entity.